

文本分类：从传统方法到大模型

技术演进 · 工程实践 · 场景落地

总览：技术演进与选型决策

技术演进时间线

时代	代表方法	核心特征	典型效果
2000s: 规则/词典	人工规则、关键词匹配	完全依赖人工，无学习能力	准确率低，维护成本高
2010s: 传统ML	TF-IDF + SVM / NB	手工特征工程 + 经典分类器	准确率 80%~85% (情感分类)
2015+: 神经网络	CNN / LSTM + 词向量	自动学习特征表示	准确率 85%~90%
2018+: 预训练微调	BERT 全量微调	预训练+微调范式，当前主流	准确率 93%~97%
2023+: 大模型	Prompt / Few-shot	零标注、生成式分类	零样本即达 85%+, few-shot 90%+

演进趋势：从人工特征 → 自动特征 → 预训练知识 → 通用生成能力。

具体示例：同一句"这个手机真的太好用了"，不同时代方法的处理方式：

- 规则时代：**匹配关键词"好"→ 正面（但"太好用了"中的"太"可能被忽略）
- 传统ML：**TF-IDF 提取特征 → SVM 分类（"太好用"作为一个整体未被建模）
- 神经网络：**词向量 + TextCNN（捕捉到"太好用"的局部模式）
- BERT微调：**上下文编码，理解"太...了"是强调语气 → 正面
- 大模型：**直接理解语义，零样本也能判断为正面

选型决策树

- └ 有标注数据 (≥ 500 条/类) + 延迟要求 < 100ms
 - └ ☒ BERT 全量微调
 - └ 示例：电商评论情感分类，已有5万条标注数据，线上要求50ms内响应
- └ 标注数据少 (< 100 条/类) 或 类别频繁新增
 - └ ☒ 大模型 Few-shot Prompt
 - └ 示例：新上线一个"用户反馈分类"需求，每类只有20条样本，类别每月新增
- └ Agent 路由 / 高频调用 (QPS > 100)
 - └ ☒ 轻量分类模型 (BERT-base / 蒸馏模型)
 - └ 示例：多Agent系统的意图路由，每秒需要处理1000+次请求，延迟<20ms
- └ 快速验证业务可行性，无任何标注
 - └ ☒ 大模型 Zero-shot，验证后收集标注

Part 1 文本分类应用场景

一、什么是文本分类

1.1 任务定义

文本分类是自然语言处理（NLP）中的一项基础任务，其核心目标是将自然语言文本映射到预定义类别空间中。

基本流程：

输入：文本序列 → 模型 → 输出：类别标签（单标签 / 多标签）

直观理解：想象一个"智能分拣员"——你给它一封信，它看完后告诉你这是"投诉信"还是"咨询信"。

类别示例：

- 情感类别：正面 / 负面 / 中性
- 主题类别：体育 / 科技 / 财经 / 娱乐
- 意图类别：查询 / 投诉 / 咨询 / 退款
- 风险等级：高 / 中 / 低

具体输入输出示例：

输入文本	分类任务	输出标签
"这个手机续航太差了，一天两充"	情感分类	负面
"央行今日降息0.25个百分点"	主题分类	财经
"我想退款，商品与描述不符"	意图分类	退款
"该用户频繁发布违规内容"	风险分类	高风险

1.2 核心能力

核心能力	说明	示例
语义理解	模型需要理解文本的真实含义，而非仅仅匹配关键词	"不太行"≠"不行"（否定语气），"好不热闹"≠"不热闹"（反语）
分布泛化	模型在训练分布之外的数据上仍能做出合理判断	训练集只有"手机"评论，测试时来了"耳机"评论仍能分类

💡 新手注："分布"是什么？

"分布"指的是数据的统计特征——哪些样本多、哪些少、特征怎么分布。训练集的分布就是"模型见过的世界"。分布泛化好 = 模型在没见过的数据上也能做对。这是机器学习最核心的难题之一：模型本质是在"记住"训练数据的模式，我们希望它学到的是"通用规律"而非"死记硬背"。

语义理解的三个难点示例：

难点1：同义表达

"这部电影很精彩" vs "这部片子真不错" → 语义相同，用词不同

难点2：否定语气

"不错" → 正面（否定词+负面词=正面）

"不太好" → 偏负面

"不能说好" → 负面

同样是否定词，搭配不同含义截然不同

难点3：一词多义

"我在苹果上班" → 公司（科技类别）

"我买了一个苹果" → 水果（食品类别）

同一个"苹果"，在不同上下文中含义完全不同

1.3 核心挑战

挑战	说明	数值示例
语义理解	同义表达、否定语气、一词多义	模型看到"还行吧"，是正面还是中性？人工标注一致率仅65%
类别不均衡	头部类别样本远多于尾部类别	电商评论：正面5000条，负面2000条，中性200条→模型偏向正面
标注成本	高质量标注耗时费力	10万条数据需3个标注员花2周标注，人力成本约3万元

二、传统业务场景

2.1 典型场景详解

场景1：内容审核

业务描述：对用户生成的内容（UGC）进行合规性检查，过滤违规内容。

输入："免费领取iPhone15，点击链接！！！”

模型判断 → 垃圾广告 → 自动删除/标记审核

特殊要求：漏检率需极低（宁可误杀不可漏放），因此需要置信度输出 + 人工复核机制。

场景2：意图识别

业务描述：在对话系统中，判断用户想做什么，从而触发对应的处理流程。

用户说："我的快递怎么还没到"

模型判断 → 物流查询意图 → 触发物流查询流程

用户说："这个商品是正品吗"

模型判断 → 商品咨询意图 → 触发商品信息查询流程

注意：意图识别通常是多标签问题——用户一句话可能同时包含多个意图。

场景3：情感分析

业务描述：判断文本的情感极性，用于舆情监控、产品评价分析等。

输入："这家餐厅菜品不错，但服务态度太差了"

→ 食物：正面，服务：负面（细粒度情感分析）

场景4：文档分类

业务描述：将文档自动归类到预定义的主题类别中。

输入："OpenAI发布GPT-5，多模态能力大幅提升"

模型判断 → 科技类（同时涉及商业类 → 多标签）

场景5：客服质检

业务描述：对客服与用户的对话记录进行自动化质量检查，评估客服服务态度、响应规范性、问题解决率等。

对话记录：

用户："这个订单我都退了一个星期了还没到账，你们怎么回事！"

客服："先生您好，退款一般是3-5个工作日到账哦，请您再耐心等待一下"

用户："我已经等了一周了！每次问都是这句话！"

质检模型判断：

→ 客户情绪：愤怒（0.92）

→ 客服回应：标准话术推诿（0.85）

→ 问题解决：未解决（0.95）

→ 质检等级：不合格（需人工复核）

与常规情感分析的区别：

维度	常规情感分析	客服质检
分析对象	单条文本	多轮对话（客服+用户双方）
评估目标	发帖人的情感极性	客服服务质量 + 用户满意度
类别体系	正面/负面/中性	情绪识别 + 服务评价 + 解决率判断
标签特点	通常单标签	多维度多标签（每轮对话打多个标签）
关键难点	反语、语气理解	谁说的（区分用户vs客服）、对话上下文

关键要求：

- 角色区分：同一段对话需要区分"用户说的"和"客服说的"，分别打标签
- 上下文依赖：客服的"好的马上处理"在第一轮是"承诺动作"，在第三轮就是"空头支票"
- 时序关系：用户情绪是否得到安抚 → 客服回应是否有效，这是一个序列建模问题
- 结果归因：问题最终解决了吗？需要看最后一轮客服的承诺是否兑现

常见质检维度：

对话片段 → 提取多个维度标签：

[用户情绪]	愤怒 / 失望 / 中性 / 满意	← 多选一
[客服态度]	热情 / 标准 / 敷衍 / 恶劣	← 多选一
[响应时效]	及时 / 延迟	← 二分类
[问题解决]	已解决 / 未解决 / 升级处理	← 多选一
[违规情况]	无违规 / 抢话 / 诱导 / 辱骂	← 多标签（可同时存在）

技术方案推荐：由于需要同时输出多个维度的标签，推荐**多任务学习**架构——共享 BERT Encoder，每个维度挂一个独立的分类头，联合训练。

各场景延迟敏感度分析

场景	用户在线等待?	延迟要求	可接受延迟	原因
意图识别	☑ 是（对话在线）	极高	< 50ms	用户发消息后等着回复，超过 500ms 对话出现明显卡顿感
内容审核	△ 部分在线	高	< 200ms	用户发帖后通常需要即时看到帖子，但审核失败可异步告知
客服质检	✗ 否（离线分析）	低	< 30 分钟	质检是对话结束后的 事后分析 ，管理人员查看报告，不直接面对用户
情感分析	✗ 否（可离线）	中	< 5 分钟	通常用于舆情大盘、商品评价统计，批量处理，用户不感知延迟
文档分类	✗ 否（可离线）	最低	< 1 小时	后台文档归档索引，延迟几十分钟或几小时完全可接受

核心判断逻辑：

用户在线等着结果 → 必须毫秒级响应（< 50ms）	→ 意图识别
用户提交后能接受短暂等待 → 秒级以内	→ 内容审核
事后分析、不直接面对用户 → 分钟级甚至小时级均可接受	→ 其余三个场景

对技术选型的影响：

延迟敏感度	推荐方案	不推荐方案
高敏感（意图识别）	BERT 蒸馏模型 / 小模型	大模型 API（500ms+ 无法接受）
中敏感（内容审核）	BERT 微调 / 轻量模型	同上
低敏感（质检/情感/文档）	大模型 API 也可接受	但 BERT 微调仍是最优解（成本低 60 倍）

💡 **新手注：**为什么同样是 BERT 微调，低敏感场景也可以用大模型 API？

响应快慢不只是模型本身决定的，还有**调用方式**。低敏感场景通常是**离线批量处理**——比如每天凌晨所有历史对话集中跑一遍质检，跑 1 小时和跑 10 分钟对用户体验没区别。这时候用大模型 API（不需要维护模型）在工程成本上有优势。但 BERT 微调的成本优势太大了（1/60），所以只要你能维护模型，BERT 仍然是更好的选择。

2.2 传统场景的共同特点

1. **类别预先定义，数量有限**：通常从几类到几百类不等
2. **大量标注数据可获取**：支持监督学习范式
3. **对推理延迟有要求**：需轻量高效的部署方案
4. **模型行为需可预期**：便于线上审计和调试

💡 新手注："监督学习范式"是什么？

机器学习三大范式：**监督学习**（给模型"输入→正确答案"对，让它学映射）、**无监督学习**（只给输入没答案，让模型自己找规律）、**强化学习**（给模型动作的奖励信号，让它学策略）。文本分类几乎都是监督学习——你需要先告诉模型"这句话是正面/负面"，它才能学会判断。

💡 新手注："推理延迟"为什么重要？

推理 = 模型做一次预测的过程。延迟 = 从输入到输出花了多少毫秒。线上服务对延迟极度敏感——用户发一条消息，如果500ms还没回复，体验就很差。BERT推理约10ms，大模型推理约500ms~1500ms，差了50~150倍。

为什么延迟要求高？——计算一笔账：

假设：每天100万次请求，高峰期QPS约500

方案A：BERT微调，单次推理10ms

→ 单卡QPS约100，需要5张卡 → 月成本约2万元

方案B：大模型API，单次推理500ms

→ 单次API调用0.01元

→ $100\text{万次} \times 0.01\text{元} = 1\text{万元/天} = 30\text{万元/月}$

→ 且高峰期QPS 500需要并发调用，延迟无法满足

三、大模型 Agent 场景中的文本分类

3.1 Agent 架构中的分类角色

在多 Agent 系统中，文本分类模型扮演"调度器"和"守门员"的角色：

用户输入 → 分类模型判断意图 → 路由至搜索/代码/数据 Agent

具体应用示例——智能客服系统的Agent路由：

用户："帮我查一下订单202401001的物流信息"

↓

分类模型：意图 = 物流查询，置信度 0.95

↓

路由至 → 物流查询Agent（专门处理物流相关请求）

用户："这段代码为什么报错？`def add(a, b) return a + b`"

↓

分类模型：意图 = 代码问题，置信度 0.92

↓

路由至 → 代码助手Agent（专门处理编程问题）

3.2 为什么用分类模型而非让大模型直接判断？

优势	说明	数值对比
延迟低	轻量分类模型 < 20ms	BERT-base: ~10ms vs GPT-4: ~1500ms
成本低	推理成本相差2~3个数量级	本地推理: ~0.0001元/次 vs API: ~0.03元/次
行为可控	输出固定枚举类别，不会出现幻觉	100%输出合法类别 vs 大模型可能输出"我觉得这是..."
易监控	置信度可记录，便于线上灰度和调优	置信度0.3→走大模型兜底

关键洞察：在 Agent 系统中，分类模型是高频调用的"基础设施"，必须轻量、快速、可控。一个典型的多Agent系统，分类模型每天可能被调用数百万次，而大模型只需处理分类模型不确定的少量请求。

四、任务分类维度

4.1 按标签数量分

类型	说明	输出差异	示例
单标签分类	每条文本属于且仅属于一个类别	Softmax（概率之和为1）	新闻只属于"体育"或"科技"
多标签分类	一条文本可属于多个类别	Sigmoid（每类独立概率）	新闻同时属于"科技"和"商业"

💡 **新手注：Softmax 和 Sigmoid 的本质区别？**

- **Softmax：**所有类别的概率做归一化，加起来=1。本质是"多选一"——如果"体育"概率变高，"科技"就必须变低，它们是竞争关系。
- **Sigmoid：**每个类别独立算一个概率，各自在0~1之间，互不影响。本质是"多选多"——"科技"概率0.9和"商业"概率0.8可以同时成立。
- **选错后果：**如果多标签任务用了Softmax，模型被迫只选一个类别，会丢失信息。

Softmax vs Sigmoid 数值示例：

单标签分类（Softmax）——3个类别互斥，概率之和为1：

正面：0.7，负面：0.2，中性：0.1 → 预测：正面

多标签分类（Sigmoid）——每个类别独立判断，概率之和不为1：

科技：0.9，商业：0.8，体育：0.1 → 预测：科技、商业

4.2 按文本粒度分

类型	说明	示例
句子级	最常见，输入通常 < 512 tokens	"这部电影太好看了" → 正面
段落/文档级	需截断或层次化编码	一整篇新闻文章 → 分类为"财经"
跨句推断	需捕捉段落间的语义关系	前提: "小明今天生病了", 假设: "小明今天去上学了" → 矛盾

4.3 按数据场景分

类型	说明	典型策略	示例
全监督	充足标注数据，微调预训练模型	标准 BERT 微调	5万条标注数据，每类500+
少样本	每类仅几十条	数据增强或 few-shot learning	每类只有30条，用EDA扩充到300条
零样本	新类别无标注	依赖大模型或向量检索	新增"环保"类别，无标注数据，用大模型prompt分类

Part 2 传统文本分类方法

一、特征工程 + 经典分类器

1.1 处理流程

原始文本 → 预处理 → 特征提取 → 分类器 → 类别标签

💡 **新手注：“特征”到底是什么？**

特征 = 把文本转换成数字的方式。计算机只认数字，不认文字。特征提取就是把"这家店服务太差了"变成一个数字向量，让分类器能处理。好的特征能区分不同类别，差的特征会让模型"看不到"关键信息。

💡 **新手注：“特征工程”为什么是“工程”？**

传统方法中，特征不是模型自动学的，而是人工设计、组合、调试出来的——就像工程项目一样，需要反复试验哪些特征有用、哪些没用。这就是"工程"二字的由来。深度学习之所以革命性，就在于它用端到端训练替代了手工特征工程。

完整示例：

原始文本: "这个手机质量很好, 非常满意"

第1步: 预处理

分词 → ["这个", "手机", "质量", "很好", "非常", "满意"]

去停用词 → ["手机", "质量", "很好", "非常", "满意"]

第2步: 特征提取 (TF-IDF)

手机: 0.35, 质量: 0.42, 很好: 0.28, 非常: 0.15, 满意: 0.51

第3步: 分类器

输入特征向量 → SVM/朴素贝叶斯 → 输出: 正面

1.2 经典分类器详解

(1) 朴素贝叶斯 (Naive Bayes)

核心思想: 基于贝叶斯定理 + 条件独立假设。

贝叶斯定理:

$$P(\text{类别}|\text{文本}) = \frac{P(\text{文本}|\text{类别}) \cdot P(\text{类别})}{P(\text{文本})}$$

条件独立假设:

$$P(\text{文本}|\text{类别}) = \prod_i P(w_i|\text{类别})$$

通俗解释: 假设每个词的出现概率相互独立, 那么"这篇文本属于正面"的概率 = "手机"在正面类出现的概率 × "满意"在正面类出现的概率 × ... × 正面类的先验概率。

数值示例:

训练数据统计:

正面类(100条): "满意"出现80次 → $P(\text{满意}|\text{正面}) = 80/1000 = 0.08$

负面类(100条): "满意"出现 5次 → $P(\text{满意}|\text{负面}) = 5/800 = 0.00625$

$P(\text{正面}) = 100/200 = 0.5$, $P(\text{负面}) = 100/200 = 0.5$

测试文本: "满意"

$P(\text{正面}|\text{满意}) \propto P(\text{满意}|\text{正面}) \times P(\text{正面}) = 0.08 \times 0.5 = 0.04$

$P(\text{负面}|\text{满意}) \propto P(\text{满意}|\text{负面}) \times P(\text{负面}) = 0.00625 \times 0.5 = 0.003125$

归一化: $P(\text{正面}|\text{满意}) = 0.04/(0.04+0.003125) \approx 0.927 \rightarrow$ 预测为正面 ✓

优点	缺点
训练速度极快, 时间复杂度为 $O(N)$	条件独立假设过强 (词与词之间实际有依赖关系)
小数据集上表现稳定	忽略词序信息 ("不好"与"好不"被同等对待)
天然支持多分类	无法处理词序对语义的影响

独立假设为何不合理? ——示例:

"不好"和"好不"在朴素贝叶斯看来完全相同（都包含"不"和"好"）
但实际含义：
"不好" → 负面
"好不热闹" → 正面（"好不"是强调用法）

朴素贝叶斯无法区分这种差异，因为忽略了词序

(2) 支持向量机 (SVM)

核心思想：寻找最大间隔超平面，使不同类别之间的间隔最大化。

直观理解：想象桌上有红色球和蓝色球，SVM要找一条线把它们分开，并且让这条线离最近的球尽可能远（最大间隔）。

关键概念：

- 最大间隔：**SVM 不仅要正确分类，还要使决策边界离最近的样本点尽可能远
- 支持向量：**距离决策边界最近的那些样本点，它们决定了边界的位置
- 核函数：**通过核技巧将数据映射到高维空间，从而实现非线性分类

核函数示例：

线性核：直接在原始特征空间中找超平面
适用于：**TF-IDF**等高维稀疏特征
示例：文本分类中，特征维度几万维，线性核效果通常就很好

RBF核：将数据映射到无穷维空间
适用于：非线性可分数据
示例：二维空间中环形分布的数据，线性核无法分开，**RBF**核可以

优点	缺点
在高维稀疏特征（如 TF-IDF）上表现稳定	对特征工程质量敏感
核函数扩展了非线性分类能力	训练时间在大规模数据上较长（ $O(n^2 \sim n^3)$ ）
通过间隔最大化，泛化能力强	输出不具备概率解释（需额外标定）

(3) Logistic 回归

核心思想：线性分类器，通过 Sigmoid 函数将线性组合映射为概率。

$$P(y = 1|x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

数值示例：

假设权重 $w = [0.3, -0.5, 0.8]$ ，偏置 $b = -0.1$
输入特征 $x = [1.0, 0.5, 0.2]$ ("好"的TF-IDF=1.0, "差"=0.5, "满意"=0.2)

线性组合: $z = w^T x + b = 0.3 \times 1.0 + (-0.5) \times 0.5 + 0.8 \times 0.2 + (-0.1)$
 $= 0.3 - 0.25 + 0.16 - 0.1 = 0.11$

Sigmoid输出: $P = 1/(1+e^{(-0.11)}) = 1/(1+0.896) \approx 0.527$

→ 有52.7%的概率是正面 → 预测为正面

优点	缺点
输出概率可解释，便于业务理解	表达能力受限于线性决策边界
训练速度快，支持大规模数据	对非线性特征需要手动构造交叉特征
可作为 BERT 分类头的基础形式 (Linear + Softmax)	—

二、特征工程

2.1 Bag-of-Words (词袋模型)

核心思想：统计每个词在文本中出现的次数，构成一个向量。完全忽略词序。

示例：

语料库词汇表: ["手机", "质量", "好", "差", "满意", "退货"]

文本A: "手机质量好满意" → 向量 [1, 1, 1, 0, 1, 0]

文本B: "手机质量差退货" → 向量 [1, 1, 0, 1, 0, 1]

问题: "好手机质量差" → 向量 [1, 1, 1, 1, 0, 0]

与"差手机质量好"完全相同！但语义相反。

2.2 TF-IDF

核心思想：词频 (TF) 衡量词在当前文本中的重要性，逆文档频率 (IDF) 衡量词在整个语料中的区分力。

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w)$$

$$\text{IDF}(w) = \log \frac{N}{\text{DF}(w)}$$

其中 N 是文档总数， $\text{DF}(w)$ 是包含词 w 的文档数。

💡 **新手注：TF-IDF 的直觉**

TF-IDF 回答一个问题: "这个词在这篇文章里有多重要?"

- 光看词频(TF)不够——"的"每篇文章都出现很多次，但毫无区分力
- IDF 的作用就是惩罚"到处都出现的词"，奖励"只在少数文章出现的词"
- 两个乘在一起：一个词既在这篇文章里频繁出现，又在整个语料中很少见 → 才是真正的关键词

数值示例：

语料库共1000篇文档

词"的"：出现在990篇文档中
 $IDF = \log(1000/990) = 0.01 \rightarrow$ 区分力极低（几乎所有文档都有）

词"量子计算"：出现在10篇文档中
 $IDF = \log(1000/10) = 2.0 \rightarrow$ 区分力高（只有特定主题才有）

$\rightarrow$ TF-IDF自动降低了"的"这类常见词的权重，提升了"量子计算"这类区分性强的词的权重

2.3 N-gram

核心思想：将连续的N个词作为一个整体特征，部分保留局部语序。

示例：

原文："手机质量不好"

Unigram (1-gram): ["手机", "质量", "不好"]

Bigram (2-gram): ["手机质量", "质量不好"]

Trigram (3-gram): ["手机质量不好"]

优势：Bigram "质量不好" 能够捕捉到"不好"是修饰"质量"的，比单独的"不好"信息更丰富
代价：N-gram的特征维度随N增大呈指数增长（组合爆炸）

2.4 特征方法对比

特征方法	说明	优缺点
Bag-of-Words	统计词频，忽略词序	简单高效，但丢失语序信息
TF-IDF	词频-逆文档频率，突出有区分力的词	比纯词频更好，但仍忽略语序
N-gram	提取连续的 N 个词作为特征	部分保留局部语序，但特征维度爆炸
手工特征	否定词、情感词典、句法特征	领域依赖强，迁移性差

三、传统方法的局限与现状

3.1 核心局限（详细示例）

局限1：特征工程依赖强

在"电商评论"上设计的特征（如"好评率"、"物流关键词"）
 $\rightarrow$ 换到"新闻分类"场景，这些特征完全无用
 $\rightarrow$ 需要重新设计特征：新闻关键词、主题词、命名实体...
 $\rightarrow$ 每换一个领域，特征工程从头来过

局限2：语义理解能力弱

TF-IDF向量空间中:

"好棒" → [0, 0, 1, 0, 0, ...] ("好棒"是1个词)
"非常棒" → [0, 1, 0, 0, 1, ...] ("非常"+"棒"是2个词)

这两个向量几乎没有重叠!但语义非常相似。

而在词向量空间中:

"好棒" → [0.23, -0.15, 0.67, ...]
"非常棒" → [0.21, -0.13, 0.64, ...]
→ 距离很近,因为语义相似

局限3: 迁移能力差

在"电影评论"上训练的SVM模型 → 直接用在"餐厅评论"上

原因: 两领域的词分布不同, 特征空间不匹配

结果: 准确率从92%降到约60%

对比: BERT在"电影评论"上微调后 → 用在"餐厅评论"上

结果: 准确率仅下降3~5%, 因为有通用语言理解能力

3.2 传统方法仍有价值的场景

1. **极低资源场景**: 无 GPU、只有几百条样本
2. **强可解释性要求**: 需要知道哪个词影响了判断 (NB天然输出每个词的贡献)
3. **规则+模型混合系统**: 作为基线对照
4. **快速冷启动**: 验证业务可行性 (1小时即可跑通baseline)

3.3 学习传统方法的意义

传统方法在绝大多数场景已被神经网络取代。理解传统方法的思路, 有助于理解深度学习改进了哪些核心问题:

- 深度学习用**稠密向量**替代了稀疏特征, 解决了语义理解弱的问题
- 深度学习用**端到端训练**替代了手工特征工程, 解决了迁移能力差的问题
- 深度学习用**预训练+微调**范式, 进一步降低了标注数据的需求

Part 3 神经网络文本分类

一、从静态词向量到上下文表示

神经网络的核心突破: 将词映射为稠密低维向量, 使语义相似的词在向量空间中彼此接近。

与稀疏特征的关键区别:

💡 新手注: "稀疏"vs"稠密"到底是什么意思?

- **稀疏向量**: 几万维, 绝大部分位置是0。比如词表10万个, "手机"只在第3个位置是1, 其余99999个位置全是0。浪费存储, 且两个词向量几乎不正交就无法计算相似度。
- **稠密向量**: 几百维, 每个位置都是实数。比如"手机"→[0.23, -0.15, 0.67, ...], 每个维度都编码了某种语义信息。
- **关键优势**: 稠密向量可以用距离/余弦相似度衡量语义接近程度, 稀疏向量做不到 (因为两个不同词的向量几乎不重叠, 余弦相似度=0)。

TF-IDF（稀疏）：每个词对应一个几万维的向量，只有1~2个位置有值
"手机" → [0, 0, 1, 0, 0, 0, ..., 0] （第3位是1，其余为0）
"电话" → [0, 0, 0, 0, 1, 0, ..., 0] （第5位是1）
余弦相似度 = 0（完全不同的位置）

词向量（稠密）：每个词对应一个低维（如300维）的实数向量
"手机" → [0.23, -0.15, 0.67, ..., 0.12]
"电话" → [0.21, -0.13, 0.64, ..., 0.11]
余弦相似度 = 0.95（语义相近，向量也相近）

二、静态词向量：Word2Vec / GloVe

2.1 核心理念

每个词对应唯一的稠密向量，语义相近的词在向量空间中聚集在一起。

💡 **新手注：词向量是怎么训练出来的？**

核心理念很简单：**让出现在相似上下文中的词，向量也相似**。训练不需要任何标注数据，只需要大量无标注文本。模型通过"预测上下文"或"预测中心词"这个自监督任务，自动学到词的语义表示。这就是为什么 Word2Vec 被称为"无监督"方法——它不需要人工标注。

经典示例（词向量算术）：

国王 - 男 + 女 ≈ 女王

数值示意：

假设4维向量（实际通常是300维）：

国王 → [0.9, 0.8, 0.1, 0.3]
男人 → [0.5, 0.6, 0.1, 0.2]
女人 → [0.5, 0.6, 0.9, 0.8]

国王 - 男人 + 女人 = [0.9, 0.8, 0.9, 0.9]
女王 → [0.9, 0.8, 0.9, 0.8] ← 非常接近！

解释：

第1、2维编码了"皇室"属性 → 国王和女王都有
第3、4维编码了"性别"属性 → 减去"男"加上"女"实现了性别转换

2.2 两种训练方式

方法	核心理念	特点
CBOW	用上下文词预测中心词	适合小数据集，训练快
Skip-gram	用中心词预测上下文词	适合大数据集，向量质量更好

CBOW 示例：

句子："我 喜欢 在 周末 去 公园 散步"
窗口大小 = 2（前后各取2个词）

Step 1: 输入 —— 上下文词的向量求平均

上下文词：[我，喜欢，去，公园]
→ 每个词查词表拿向量
→ 四个向量加起来取平均 → 得到一个"上下文向量"

Step 2: 输出 —— 预测中心词是"在"
用上下文向量 → **softmax** → 输出词表中每个词的概率
→ 希望"在"的概率最大

Step 3: 反向更新
交叉熵损失 → 梯度反向传播 → 更新所有词的向量

输入（上下文）：[我，喜欢，去，公园]
目标词：在
训练目标：给定上下文，预测中心词"在"

Skip-gram 示例：

句子："我 喜欢 在 周末 去 公园 散步"
中心词："周末"

生成训练对：
(周末，在)，(周末，去)，(周末，公园)，(周末，散步)
训练目标：给定中心词"周末"，预测上下文词

2.3 核心局限

一词一义问题：每个词只有唯一的向量表示，无法处理多义词。

"bank" 的向量 = 银行的含义 + 河岸的含义 混在一起

"I went to the bank to deposit money" → 这里的bank是"银行"
"I sat by the river bank" → 这里的bank是"河岸"

但word2vec给"bank"只有一个向量，无法区分这两种含义

这就是BERT要解决的问题——上下文相关的词向量！

三、TextCNN：局部特征提取

3.1 核心机制

TextCNN 使用多尺度卷积核并行提取局部 n-gram 特征，再通过 MaxPool 聚合。

输入文本 → 词向量矩阵 → 多尺度卷积核(2/3/4-gram) → MaxPool → 拼接 → 全连接 → 类别

💡 **新手注：CNN 做文本分类，"卷积"到底在卷什么？**

CNN 原本用于图像，在图像上卷积是"看一小块区域的像素模式"。在文本上，词已经被映射为向量，卷积就是在**词向量矩阵上滑动**——每次看连续2个/3个/4个词的向量，提取出一个"局部特征值"。本质上，2-gram卷积核在学习"二元短语模式"，3-gram在学习"三元短语模式"，和 N-gram 的思想一致，但由模型自动学习权重，而非人工统计。

💡 新手注：MaxPool 为什么只取最大值？

一句话中不是每个词都重要——"手机 质量 很好 非常 满意"，"很好"和"满意"是关键，"非常"是修饰。MaxPool 在每个卷积核的所有输出中只保留最大的那个 → 只保留最显著的语义信号，忽略无关词的噪声。这叫"平移不变性"——关键特征出现在句子任何位置都能被捕捉到。

详细示例：

输入："手机 质量 很好 非常 满意"

↓

词向量矩阵（假设4维）：

手机	[0.2, -0.1, 0.5, 0.3]	
质量	[0.1, 0.4, 0.2, 0.1]	
很好	[0.6, 0.3, 0.8, 0.7]	← 关键特征！
非常	[0.3, 0.2, 0.1, 0.4]	
满意	[0.7, 0.5, 0.9, 0.6]	← 关键特征！

↓

2-gram卷积核（滑窗大小2）

"手机质量"	→ 卷积 → 特征值	
"质量很好"	→ 卷积 → 特征值	← 重要！
"很好非常"	→ 卷积 → 特征值	
"非常满意"	→ 卷积 → 特征值	← 重要！

↓

3-gram卷积核（滑窗大小3）

"手机质量很好"	→ 卷积 → 特征值	
"质量很好非常"	→ 卷积 → 特征值	← 重要！
"很好非常满意"	→ 卷积 → 特征值	

↓

MaxPool（每个卷积核取最大值）

保留最显著的特征激活

↓ 拼接所有卷积核的输出 → 全连接层 → Softmax → 类别

3.2 关键设计

组件	说明	为什么这样设计
多尺度卷积核	2/3/4-gram并行	不同尺度捕捉不同粒度的特征：2-gram捕捉短语，4-gram捕捉短句
MaxPool	对每个卷积核取最大值	只保留最显著的特征，忽略无关词，实现平移不变性
拼接	拼接不同卷积核的输出	将多尺度特征融合，形成综合表示

3.3 优缺点

优点	缺点
擅长短文本分类	无法捕捉长距离依赖（如"虽然...但是..."中的转折关系）
推理速度极快（GPU上<1ms）	对词序的建模有限（卷积只能捕捉局部模式）
实现简单（100行PyTorch代码）	缺乏全局语义理解

当前地位：工程兜底方案——在需要极低延迟的场景下仍被使用。

四、BiLSTM：序列建模

4.1 核心机制

双向 LSTM 同时从左→右、右→左扫描序列，融合前后文信息后进行分类。

输入文本 → 词向量 → 前向LSTM(左→右) + 后向LSTM(右→左) → 拼接 → 全连接 → 类别

为什么需要双向？——示例：

"这个手机 [不] 好"

前向LSTM看到"好"时，还不知道后面有"不"

→ 如果只看前向，可能误判为正面

"好 [不] 这个手机"（倒序）

后向LSTM从右往左看，先看到"手机"，再看到"不"，再看到"好"

→ 能正确理解"不好"的含义

双向结合：前向+后向的信息融合后，能同时理解"好"和"不"的修饰关系

4.2 LSTM 单元关键组件

组件	公式	作用
遗忘门	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	决定丢弃哪些历史信息（0=丢弃，1=保留）
输入门	$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	决定更新哪些新信息
输出门	$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	决定输出哪些信息
细胞状态	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	信息传递的"高速公路"，缓解梯度消失

💡 **新手注：LSTM 的"门"到底是什么？**

"门"就是一个0~1之间的权重向量，控制信息的通过量。**0=完全关上（丢弃）**，**1=完全打开（保留）**，**0.5=通过一半**。三个门各司其职：

- **遗忘门：**旧信息哪些要忘？读到"不"时，遗忘门会调高，开始"遗忘"之前的正面倾向
- **输入门：**新信息哪些要存？读到关键情感词时，输入门会打开，存入新信息
- **输出门：**当前状态哪些要输出给下一步？决定隐藏状态h的内容

💡 **新手注：为什么 LSTM 能缓解梯度消失？**

普通RNN中，信息每一步都要乘一个权重，经过很多步后信号就衰减到接近0（梯度消失）。LSTM的细胞状态C有一条"加法通道" ($C_t = f_t \odot C_{t-1} + \dots$)，信息可以几乎无损地传递下去，只要遗忘门接近1，旧信息就不会被抹掉。这就是"高速公路"的含义。

LSTM处理否定句的示例：

输入序列: "这家 餐厅 服务 不 好"

t=1: "这家" → 遗忘门≈1（保留），输入门≈0.3（少量更新）→ 状态更新

t=2: "餐厅" → 遗忘门≈1（保留），输入门≈0.5 → 累积语义

t=3: "服务" → 遗忘门≈1（保留），输入门≈0.7 → 关键信息！

t=4: "不" → 遗忘门≈0.8（部分遗忘之前的正面倾向），输入门≈0.9 → 关键！

t=5: "好" → 但"不"已经修改了上下文，最终理解为"不好"→ 负面

4.3 优缺点

优点	缺点
能捕捉序列中的前后文依赖	长距离依赖随序列加长而衰减 (>200 token后效果下降)
双向建模，语义理解更完整	训练速度慢（无法并行化，必须逐token计算）
适用于序列标注任务（NER/POS）	参数量较大

当前地位：逐渐被 Transformer 取代——但仍用于对序列顺序敏感的任务。

五、Transformer：全局注意力

5.1 核心机制

Self-Attention 让每个 token 直接关注序列中任意位置，天然并行化。

Attention(Q, K, V) = softmax (QK^T / sqrt(d_k)) V

💡 新手注：Q/K/V 到底是什么？用图书馆类比

想象你去图书馆找书：

- Q (Query, 查询) = 你想找什么书 ("我要一本讲二战的历史书")
- K (Key, 键) = 每本书的标签 ("历史·二战"、"科幻·太空".....)
- V (Value, 值) = 书的实际内容

流程：你的Q和每本书的K做比较（点积）→ 哪些书和你的需求最匹配，权重最高 → 用权重对V加权求和 → 得到"融合了最相关信息的结果"。

在文本分类中：每个词都有自己的Q/K/V, "不好"这个词的Q会去和其他词的K比较 → 发现"好"和它最相关 → 加权融合"好"的V → 理解了"不好"的真实含义。

💡 新手注：公式中为什么要除以 sqrt(d_k)？

这叫**缩放因子 (Scale)**。当维度 d_k 很大时，Q和K的点积数值会变得很大，导致Softmax输出的概率分布极端化（接近one-hot），梯度几乎为0，训练不动。除以 sqrt(d_k) 可以把点积拉回合理范围，让梯度正常流动。这就是为什么叫"Scaled Dot-Product Attention"。

逐步数值示例：

输入序列: "我 喜欢 这部 电影"（假设4维词向量）

第1步：每个词向量通过3个线性变换得到Q、K、V

我： Q=[0.1, 0.2], K=[0.3, 0.1], v=[0.5, 0.4]

喜欢： Q=[0.4, 0.3], K=[0.2, 0.5], v=[0.3, 0.6]

这部: $Q=[0.2, 0.1]$, $K=[0.1, 0.3]$, $V=[0.2, 0.1]$
电影: $Q=[0.3, 0.4]$, $K=[0.4, 0.2]$, $V=[0.6, 0.3]$

第2步: 计算注意力分数 ($Q \times K^T / \sqrt{d_k}$)

"我"对所有词的注意力分数:

我: $Q \cdot K^T = 0.1 \times 0.3 + 0.2 \times 0.1 = 0.05$

喜欢: $Q \cdot K^T = 0.1 \times 0.2 + 0.2 \times 0.5 = 0.12 \leftarrow$ "我"最关注"喜欢"

这部: $Q \cdot K^T = 0.1 \times 0.1 + 0.2 \times 0.3 = 0.07$

电影: $Q \cdot K^T = 0.1 \times 0.4 + 0.2 \times 0.2 = 0.08$

第3步: Softmax归一化

注意力权重 = $\text{softmax}([0.05, 0.12, 0.07, 0.08])$

$\approx [0.22, 0.30, 0.24, 0.24] \leftarrow$ "喜欢"权重最高

第4步: 加权求和V

"我"的新表示 = $0.22 \times V_{\text{我}} + 0.30 \times V_{\text{喜欢}} + 0.24 \times V_{\text{这部}} + 0.24 \times V_{\text{电影}}$

→ 融合了全局信息的上下文表示!

5.2 Transformer 的关键创新

创新	说明	为什么重要
Self-Attention	每个 token 可以直接关注序列中任意位置	解决了RNN逐步传递信息的瓶颈，一步到位
Multi-Head Attention	多个注意力头并行计算	不同头关注不同方面（如一个关注语法、一个关注语义）
位置编码	注入位置信息	弥补注意力机制本身无位置感知的不足
完全并行化	相比 RNN 的序列计算	训练效率大幅提升，GPU利用率高

💡 新手注：为什么 Self-Attention 没有"位置感知"?

Attention 的计算只看"谁和谁语义相关"，完全不管谁在前面谁在后面。比如"我喜欢这部电影"和"部电影这我喜欢我"，在Attention看来Q/K/V的点积结果是一样的——因为它只计算相似度，不计算顺序。位置编码就是给每个位置加一个独特的"位置向量"，让模型能区分"第1个词"和"第3个词"。

💡 新手注：Multi-Head Attention 为什么需要多个头?

一个注意力头只能学一种"关注模式"。但语言关系是多元的——同一个句子里，有语法关系（主语→动词）、修饰关系（形容词→名词）、指代关系（代词→先行词）、否定关系（"不"→被否定的词）。多个头就像多个"视角"，每个头专盯一种关系，最后拼接起来形成全面理解。

Multi-Head Attention 示例：

8个注意力头可能学到不同的模式：

Head 1: 关注语法关系（主语→动词）

Head 2: 关注修饰关系（形容词→名词）

Head 3: 关注指代关系（代词→先行词）

Head 4: 关注否定词（"不"→被否定的词）

...

每个头提供不同的视角，拼接后形成全面的理解

当前地位：主流基础架构——奠定了 BERT、GPT 等预训练模型的基础。

六、神经网络方法对比速览

方法	核心机制	擅长场景	主要局限	当前地位
TextCNN	多尺度卷积 + MaxPool	短文本、高速推理	无法捕捉长距依赖	工程兜底方案
BiLSTM	双向 RNN 序列建模	序列标注、中等文本	长距衰减、训练慢	逐渐被取代
Transformer	Self-Attention 全局建模	通用，奠定 BERT 基础	参数量大，需预训练	主流基础架构

七、演进总结

word2Vec/GloVe（静态词向量）
↓ 问题：一词一义，无法处理多义词
↓ 例："bank"只有一个向量，无法区分"银行"和"河岸"
TextCNN（局部特征提取）
↓ 问题：无法捕捉长距离依赖
↓ 例："虽然...但是..."的转折关系被割裂
BiLSTM（序列建模）
↓ 问题：长距离衰减、无法并行
↓ 例：200+ token后，开头的词对结尾影响很小
Transformer（全局注意力）
↓ 解决了上述所有问题：上下文表示 + 全局建模 + 并行化
↓ 下一步：堆叠 Transformer，在大规模语料上预训练 → BERT

Part 4 BERT 与 LLM 微调方法

一、预训练 + 微调范式

1.1 两个阶段

阶段	数据	任务	结果
阶段一：预训练	大规模无标注语料（维基百科、书籍等）	MLM（掩码语言模型）+ NSP（下句预测）	模型学到通用语言表示
阶段二：微调	少量有标注的下游任务数据（几百~几万条）	在预训练权重基础上继续训练	适配具体任务

 **新手注：**"预训练+微调"为什么是革命性范式？

之前的做法：从零开始训练模型（随机初始化权重→用标注数据训练→得到模型），需要海量标注数据。

阶段1: 收集原始文本

"东西不错"、"太差了"、"一般般"..... (只有文本, 没有标签)

↓ 人工标注

阶段2: 得到标注数据 (= 训练数据)

("东西不错", 正面)、("太差了", 负面)、("一般般", 中性)

↓ 喂给模型训练

阶段3: 模型学习

输入 "东西不错" → 模型预测 → 和正确答案"正面"对比 → 算Loss → 更新参数

↓ 反复迭代

阶段4: 模型上线

输入新文本 → 模型自己预测标签 (不再需要人标注)

新范式: 先在无标注数据上"预训练"出通用语言能力 (不用人标注, 用自监督任务), 再用少量标注数据"微调"到具体任务。本质上是把"学语言"和"学任务"解耦了——预训练阶段免费获得了语言能力, 微调阶段只需要很少的标注数据就能达到很好的效果。

原始文本:

"我今天去银行取了一些钱"

自监督任务自动生成训练对:

输入 (给模型看的)	答案	
"我今天去[?]取了一些钱"	银行	← MLM
"我 喜欢 → [?]"	去	← CBOW
"周末 → [?]"	在	← Skip-gram

自监督 = 文本自己出题考自己, 人不用参与。把"我[?]去银行"的答案设为"今", 模型反复做这种完形填空, 就免费学会了每个词的含义和句子的结构。

MLM (掩码语言模型) 示例:

原始文本: "我今天去银行取了一些钱"

掩码后: "我今天去[MASK]取了一些钱"

模型预测: [MASK] = "银行" (概率0.85)

- 模型学会了根据上下文预测被遮盖的词
- 这要求模型理解"取钱"的语境 → 推断出是"银行"而非"河岸"
- 因此BERT的词向量是上下文相关的!

💡 新手注: MLM 为什么能让模型学到上下文表示?

因为模型要猜对被遮住的词, 就必须理解周围词的含义。比如"我今天去_取了一些钱", 模型必须理解"取钱"这个语境才能推断出是"银行"而不是"河岸"。经过大量这样的训练后, 模型给"银行"的向量就不再是一个固定值, 而是会根据上下文变化——和"取钱"一起出现时, 向量偏向"金融机构"的语义; 和"河边"一起出现时, 向量偏向"河岸"的语义。这就是"上下文相关词向量"的含义。

[MASK] 不是模型提供的, 是预处理代码做的——在数据喂给模型之前就已经遮好了。

原始文本:

"我今天去银行取了一些钱"

↓ 第一步: 分词 (tokenization)

["我", "今", "天", "去", "银", "行", "取", "了", "一", "些", "钱"]

↓ 第二步：随机选 15% 的词作为"目标"
被选中的词：["银"] ← 假设随机选到了它

↓ 第三步：替换（三种方式随机选一种）

80% → 替换为 [MASK] "我今天去[MASK]行取了一些钱" # 这 80/10/10 是针对"被选中
的那 15% 的词", 做[mask]或者替换或者不变, 不是针对所有词。

10% → 替换为随机词 "我今天去 猫 行取了一些钱"

10% → 保留原词不变 "我今天去 银 行取了一些钱"

↓ 第四步：喂给模型

模型输入：["我", "今", "天", "去", "[MASK]", "行", "...]

正确答案（监督信号）："银" ← 让模型猜这个

模型看上下文 → 输出"银" → 和正确答案对比 → 算Loss → 更新参数

这不是模型内部做的事，是数据预处理（DataLoader）做的

```
def mask_tokens(input_ids):  
    """在 token 序列中随机遮住 15% 的词"""  
    labels = input_ids.clone() # 保存正确答案  
    mask = torch.rand(input_ids.shape) < 0.15 # 随机选15%  
  
    for i in range(len(input_ids)):  
        if mask[i]:  
            prob = random.random()  
            if prob < 0.8:  
                input_ids[i] = tokenizer.mask_token_id # [MASK]  
            elif prob < 0.9:  
                input_ids[i] = random.randint(0, vocab_size-1) # 随机词  
            else:  
                pass # 保留原词，不动  
  
    return input_ids, labels # (遮后的输入，正确答案)
```

NSP（下句预测）示例：

正例：

句子A: "我今天去银行取钱"

句子B: "排队的人很多" → 是下一句（IsNext）

负例：

句子A: "我今天去银行取钱"

句子B: "地球是太阳系第三颗行星" → 不是下一句（NotNext）

→ 模型学会判断两个句子是否有连贯关系

→ [CLS]向量因此被训练为聚合整句语义的表示

1.2 为什么预训练+微调有效？

原因	说明	类比
通用表示	预训练赋予模型开箱即用的语言理解能力	像一个大学毕业生，已有基础知识，只需岗前培训
参数复用	BERT 的 Encoder 权重被下游任务共享	同一个大学毕业生可以做不同岗位的工作
迁移能力	同一个 BERT 可微调用于分类、NER、问答等	学会了中文后，看新闻、读小说、写报告都行

二、BERT 用于分类的整体架构



💡 新手注：[CLS] 和 [SEP] 是什么？

这是 BERT 的两个特殊 Token：

- **[CLS]** (Classification)：永远放在句子最前面，经过12层Transformer后，它的向量会聚合整个句子的语义信息，用于分类。可以理解为"总结者"。
- **[SEP]** (Separator)：句子分隔符，标记句子边界。单句分类时放在末尾，双句任务时放在两句话之间。

💡 新手注：Token、Token ID、Segment ID、Attention Mask 分别是什么？

- **Token**：文本被分词后的最小单元。如"今天天气很好"→["今", "天", "天", "气", "很", "好"] (中文按字分词)
- **Token ID**：每个Token在词表中的编号。BERT词表约3万词，"今"可能编号791。模型输入就是这些编号。
- **Segment ID**：标记每个Token属于第几个句子。0=第一句，1=第二句。单句分类全0即可。
- **Attention Mask**：标记哪些位置是真实Token (1)，哪些是填充PAD (0)。告诉模型"PAD位置不要关注"。

各部分维度详解：

输入: [CLS] 今天 天气 很好 [SEP]
→ Token IDs: [101, 791, 3358, 2523, 102] ← tokenizer编码
→ Segment IDs: [0, 0, 0, 0, 0] ← 单句全0
→ Attention Mask: [1, 1, 1, 1, 1] ← 无padding

BERT Encoder输出:
last_hidden_state: [1, 5, 768] ← [batch, seq_len, hidden_size]
→ [CLS]向量: last_hidden_state[0][0] → 768维

分类头:
Linear(768, 3) + Softmax → [正面概率, 负面概率, 中性概率]

BERT Encoder 输出 + 分类头, 逐层拆解

整体画面

输入文本: "今天天气很好"
↓
BERT Encoder (12层Transformer, 每层有Self-Attention + FFN)
↓
last_hidden_state: [1, 5, 768]
↓
取 [CLS] 向量 → 768维
↓
Linear(768, 3) → [0.6, 0.3, 0.1]
↓
Softmax → [0.55, 0.28, 0.17]
↓
预测结果: 正面

第一步: 输入是怎么变成 5 个 token 的

原始文本: "今天天气很好"
↓ 分词
Token: [CLS] 今 天 天 气 很 好 [SEP]
↓ 映射为ID
Token ID: [101, 791, 1921, 791, 3698, 2523, 1962, 102]
↓ 查词表, 每个ID对应一个向量
向量: [768] [768] [768] [768] [768] [768] [768] [768]
↓ 拼成矩阵
输入矩阵: 8 × 768 ← 但示例显示 seq_len=5, 下面解释

注意: 文档示例 [1, 5, 768] 中的 5 只是示意数字, 实际中文分词后可能更长。这里按 5 来理解即可。

第二步: last_hidden_state: [1, 5, 768] 三维含义

[1, 5, 768]
↑ ↑ ↑
batch seq_len hidden_size

维度	含义	这个数字表示
1 (batch)	一次喂了几句话	1 句
5 (seq_len)	这句话有几个 token	5 个 token (含 [CLS] [SEP])
768 (hidden_size)	每个 token 的向量维度	768 个浮点数

可以把它想象成一张表：

	dim0	dim1	dim2	...	dim767	
[CLS]	→ 0.23	0.15	-0.08	...	0.42	← 我们要取的就是这一行！
今	→ 0.11	-0.33	0.67	...	-0.19	
天	→ -0.45	0.21	0.03	...	0.55	
很	→ 0.78	-0.12	-0.91	...	0.33	
好	→ -0.22	0.44	0.18	...	-0.67	

5 行 × 768 列 = 每个 token 一个 768 维向量。

第三步：为什么取 [CLS] 向量

last_hidden_state[0]	← batch里第0句
last_hidden_state[0][0]	← 那句的第0个token = [CLS]

[CLS] 放在句首，经过 12 层 Transformer 后，它的向量已经被所有其他 token "灌注"了信息——因为 Self-Attention 让它和句中每个词都"交流过"。

类比：[CLS] 是一个"书记员"
→ 每层 Self-Attention 都会去读每个词的信息
→ 12 层下来，书记员已经掌握了全文要点
→ 它的向量 = 全文语义的压缩表示

为什么取最后一层？

因为层数越深，语义越抽象、越"懂"整句话的意思。浅层还在看字面，深层才"看懂"语义。

- 第1层（看懂字）： "今天天气很好"——认识每个字
- 第3层（看懂词）： "天气"是一个词，"很好"是一个词组
- 第6层（看懂句）： "今天天气很好"表达一种正面状态
- 第9层（理解意图）： 说话人在表达满意/愉快
- 第12层（总结全文）： 这是一个"正面情感"的陈述 ← 取这层做分类

浅层 = 字面信息（这个词是什么），深层 = 语义信息（这句话什么含义）。

BERT 各层实际在学什么

研究（如 BERTology）发现，不同层关注的东西完全不同：

层 1~3	（浅层）：短语结构、词性、相邻词搭配
层 4~6	（中层）：句法关系、主谓宾、修饰关系
层 7~9	（中深）：局部语义消歧（多义词选择哪个含义）
层 10~12	（深层）：全局语义、句间关系、任务相关语义

分类需要的是"全文整体语义"，这正是第 12 层最擅长的。

取第几层	什么场景	原因
最后一层（默认）	文本分类、情感分析	需要全文高层语义
最后四层拼接	效果要求极高	融合多层信息，浅层补细节，深层补语义
倒数第二层	RoBERTa 微调	最后一层过度适应预训练任务，倒数第二层泛化更好
中间层（6~9 层）	词法任务（NER、分词）	句法信息在中间层最丰富

取最后一层是因为分类需要"读懂全文"，深层才懂。但不要盲信——RoBERTa 上倒数第二层更好，超参数调优时可以试试"最后4层拼接"或 Mean Pooling。

第四步：Linear(768, 3) 做了什么

这是一个全连接层，就是一个矩阵乘法：

输入：[CLS]向量 = [x₀, x₁, x₂, ..., x₇₆₇] ← 768个数

权重矩阵 W = 768行 × 3列 偏置 b = [b₀, b₁, b₂]

	w ₀₀	w ₀₁	w ₀₂	
	w ₁₀	w ₁₁	w ₁₂	
	...	...	...	
	w ₇₆₇₀	w ₇₆₇₁	w ₇₆₇₂	

计算：输出 = 输入 × W + b

O₀ = x₀ · w₀₀ + x₁ · w₁₀ + ... + x₇₆₇ · w₇₆₇₀ + b₀ ← 正面分数

O₁ = x₀ · w₀₁ + x₁ · w₁₁ + ... + x₇₆₇ · w₇₆₇₁ + b₁ ← 负面分数

O₂ = x₀ · w₀₂ + x₁ · w₁₂ + ... + x₇₆₇ · w₇₆₇₂ + b₂ ← 中性分数

输出 logits: [0.6, 0.3, 0.1]

本质：768 维的语义向量被"压缩"成 3 个分数——每个类别一个。这些分数叫 **logits**（未归一化概率，可能是任意实数）。

第五步：Softmax 把分数变成概率

logits: [0.6, 0.3, 0.1] ← 可以是任意范围

Softmax 公式：
P(第i类) = e^logit_i / (e^logit_0 + e^logit_1 + e^logit_2)

计算：
e^0.6 = 1.822
e^0.3 = 1.350

$$e^{0.1} = 1.105$$
$$\text{总和} = 4.277$$

正面概率 = $1.822 / 4.277 = 0.426$
 负面概率 = $1.350 / 4.277 = 0.316$
 中性概率 = $1.105 / 4.277 = 0.258$

输出: [0.426, 0.316, 0.258] ← 三个分数相加 = 1.0

Softmax 做的事	效果
e^n 放大差距	分数高的被放大更多
除以总和归一化	强制输出加起来 = 1
结果	可以读出"模型更倾向正面"

代码视角

```
import torch.nn as nn

# 分类头
classifier = nn.Linear(768, 3) # 768输入 → 3输出

# forward过程
cls_vector = last_hidden_state[:, 0, :] # [1, 768] ← 取[CLS]
logits = classifier(cls_vector) # [1, 3] ← [0.6, 0.3, 0.1]
probs = nn.functional.softmax(logits, dim=-1) # [1, 3] ← [0.43, 0.32, 0.26]
pred = probs.argmax(dim=-1) # 0 ← "正面"
```

一句话总结

[CLS]向量(768维) $\rightarrow$ W(768x3) $\rightarrow$ logits(3个分数) $\xrightarrow{\text{Softmax}}$ 概率(3个, 和为1)

$\uparrow$ $\uparrow$ $\uparrow$

全文语义浓缩 每个类别打分 哪个类别最可能

三、微调方式一：取 [CLS] 向量

3.1 原理

[CLS] 是 BERT 输入序列的第一个特殊 token。经过所有 Transformer 层后，其对应的隐向量被设计为聚合整个句子的语义信息，直接用于下游分类任务。

为什么 [CLS] 能聚合整句信息?

在预训练的NSP任务中:

- [CLS] 需要判断两个句子是否是上下句关系
- [CLS] 必须“看过”所有token才能做出判断
- 经过多层Self-Attention后, [CLS] 已经融合了全文信息

因此 [CLS] 的隐向量 = 整句话的语义压缩表示

3.2 完整代码示例

```
import torch
from transformers import BertTokenizer, BertForSequenceClassification

# 加载模型和分词器
tokenizer = BertTokenizer.from_pretrained('bert-base-chinese')
model = BertForSequenceClassification.from_pretrained(
    'bert-base-chinese',
    num_labels=3 # 正面/负面/中性
)

# 输入文本
text = "这家店服务真的太差了"
inputs = tokenizer(text, return_tensors='pt', padding=True, truncation=True)

# 前向推理
with torch.no_grad():
    outputs = model(**inputs)
    logits = outputs.logits # [1, 3]
    pred = torch.argmax(logits, dim=1) # 预测类别

print(f"预测类别: {pred.item()}") # 0=正面, 1=负面, 2=中性
```

3.3 适用场景与注意点

类型	说明
适用	句子级单标签分类（最常用、最简单）
注意	[CLS] 的聚合能力依赖预训练质量；NSP 任务专门训练它聚合句义
注意	对于没有 NSP 预训练的模型（如 RoBERTa），[CLS] 效果可能弱于 Pooling

四、微调方式二：Mean / Max Pooling

4.1 Mean Pooling（平均池化）

对所有有效 token 的隐向量取均值（排除 [PAD]），得到整句的平均表示。

```
# mask: [B, L], [PAD] 位为 0
hs = outputs.last_hidden_state # [B, L, 768]
m = mask.unsqueeze(-1).float() # [B, L, 1]
mean_vec = (hs * m).sum(1) / m.sum(1) # [B, 768]
```

数值示例：

句子: "很好" + [PAD] [PAD]
隐向量:
很好 → [0.6, 0.3, 0.8, ...]
[PAD] → [0.1, 0.0, 0.2, ...] ← padding, 不应参与计算

Mean Pooling (排除PAD):
= [0.6, 0.3, 0.8, ...] / 1 = [0.6, 0.3, 0.8, ...]

如果不排除PAD:
= ([0.6, 0.3, 0.8, ...] + [0.1, 0.0, 0.2, ...]) / 2 = [0.35, 0.15, 0.5, ...]
→ 被PAD稀释了! 语义信息变弱

4.2 Max Pooling (最大池化)

在每个维度上取所有 token 隐向量的最大值, 保留最显著的特征激活。

```
max_vec = outputs.last_hidden_state.max(dim=1).values # [B, 768]
```

为什么Max Pooling有效?

句子: "手机质量很好但电池太差"
隐向量 (简化为3维):
手机 → [0.2, 0.1, 0.3]
质量 → [0.1, 0.4, 0.2]
很好 → [0.8, 0.3, 0.7] ← 最显著的正向特征
但 → [0.1, 0.1, 0.1]
电池 → [0.3, 0.2, 0.4]
太差 → [0.2, 0.9, 0.1] ← 最显著的负向特征

Max Pooling: [0.8, 0.9, 0.7] ← 保留了最正向和最负向的特征!
Mean Pooling: [0.28, 0.33, 0.30] ← 特征被平均, 显著信息被稀释

4.3 [CLS] vs. Pooling 对比

维度	[CLS]	Pooling
信息来源	单一 [CLS] 向量	全部 token 均参与
预训练依赖	依赖 NSP 任务训练好的聚合能力	不依赖预训练任务设计
模型适配	BERT 原生; RoBERTa 较弱	RoBERTa / DeBERTa 上更稳健
长文本	聚合能力受序列长度影响	Mean 更充分利用全文信息
实践推荐	短文本首选, 实现最简单	长文本或语义相似度任务首选

五、微调方式三：多标签 / 多任务变体

5.1 多标签分类 (Multi-label)

场景：一篇新闻同时属于"科技"和"经济"；一条评论同时涉及"服务"和"价格"。

Softmax vs Sigmoid 输出对比：

3个类别：[科技，财经，体育]

Softmax输出（单标签，概率之和=1）：

科技：0.6，财经：0.3，体育：0.1

→ 预测：科技（只能选一个）

Sigmoid输出（多标签，每类独立）：

科技：0.9，财经：0.8，体育：0.1

→ 预测：科技、财经（可以选多个）

💡 **新手注：BCEWithLogitsLoss 是什么？**

BCE = Binary Cross Entropy（二元交叉熵），用于多标签分类。它把Sigmoid和二元交叉熵合并成一步，比先Sigmoid再算Loss数值更稳定。"WithLogits"意思是输入是未归一化的logits（不是0~1之间的概率），函数内部自动做Sigmoid。每个类别独立算一个二元分类的Loss，再求平均。

完整代码示例：

```
import torch
import torch.nn as nn

class MultiLabelClassifier(nn.Module):
    def __init__(self, bert, num_labels):
        super().__init__()
        self.bert = bert
        self.classifier = nn.Linear(768, num_labels)

    def forward(self, input_ids, attention_mask):
        outputs = self.bert(input_ids, attention_mask=attention_mask)
        cls_vec = outputs.last_hidden_state[:, 0, :]
        logits = self.classifier(cls_vec) # [B, C]
        return logits

# 损失函数
criterion = nn.BCEWithLogitsLoss()

# 预测
logits = model(input_ids, attention_mask=mask)
probs = torch.sigmoid(logits)
preds = (probs > 0.5).int() # 每个类别独立判断
```

5.2 多任务微调 (Multi-task)

思路：共享同一个 BERT Encoder，顶部接多个独立分类头，联合训练。

```

      共享 BERT Encoder
      /       |       \
主题分类头  情感分类头  风险分类头
(10类)      (3类)      (2类)
```

代码示例：

```
class MultiTaskModel(nn.Module):
    def __init__(self, bert):
        super().__init__()
        self.bert = bert
        self.topic_head = nn.Linear(768, 10) # 主题10分类
        self.sentiment_head = nn.Linear(768, 3) # 情感3分类
        self.risk_head = nn.Linear(768, 2) # 风险2分类

    def forward(self, input_ids, attention_mask, task='topic'):
        outputs = self.bert(input_ids, attention_mask=attention_mask)
        cls_vec = outputs.last_hidden_state[:, 0, :]

        if task == 'topic':
            return self.topic_head(cls_vec)
        elif task == 'sentiment':
            return self.sentiment_head(cls_vec)
        elif task == 'risk':
            return self.risk_head(cls_vec)
```

注意：任务间差异过大时，梯度可能互相干扰。例如"主题分类"和"情感分类"可能互补，但"主题分类"和"代码语言识别"差异太大，不适合一起训练。

六、全量微调 vs. 冻结微调

6.1 全量微调（推荐）

- BERT 所有层 + 分类头一起训练，无任何参数冻结
- 模型可以根据任务分布调整内部表示，效果最好
- 实践中几乎总是优于冻结微调
- 使用较小学习率（ $2e-5 \sim 5e-5$ ）防止预训练知识被遗忘
- A100 / V100 单卡可在数小时内完成 BERT-base 微调

💡 新手注：什么是"灾难性遗忘"？

模型在微调时，如果学习率太大或训练太久，会"忘记"预训练阶段学到的通用语言知识，只记住当前任务的少量数据模式。表现：训练集F1很高，但在新数据上表现很差。类比：一个人疯狂突击一个月考研数学，考完发现高中学的语文全忘了。

F1 详解——从最基础讲起

一、F1 要解决什么问题

单一指标都有盲区：

仅看 Accuracy	问题
100 条里 95 条正面，5 条负面	
模型全部预测"正面"	Accuracy = $95/100 = 95\%$ ，看起来很美
但负面 Recall = $0/5 = 0\%$	一个负面都没抓到，模型等于废物

F1 = 同时衡量"抓得全不全"和"抓得准不准"，暴露出 Accuracy 掩盖的问题。

二、四个基础指标：以"找负面评论"为例

假设测试集有 100 条评论：

实际：70 条正面，30 条负面
模型预测结果：

预测正面：68 条 → 实际正面 65 条，实际负面 3 条（误判）
预测负面：32 条 → 实际负面 27 条，实际正面 5 条（误判）

画成 **混淆矩阵**：

	预测正面	预测负面	
实际正面	65	5	← 70条正面
实际负面	3	27	← 30条负面

术语	含义	本次数字	通俗说法
TP（真阳性）	预测负面，确实是负面	27	抓对了
FP（假阳性）	预测负面，其实是正面	5	冤枉了
FN（假阴性）	预测正面，其实是负面	3	漏掉了
TN（真阴性）	预测正面，确实是正面	65	排除对了

三、精确率（Precision）——"宁缺毋滥"

$$Precision = \frac{TP}{TP+FP} = \frac{27}{27+5} = 0.844$$

模型说"这是负面"的 32 条里，有多少是真的负面？84.4%。精确率 = 抓的准不准。越高 = 越不冤枉好人。

极端例子：模型只预测了 1 条为负面，且对了 → $Precision = 1/1 = 100\%$ 。但漏了 29 条（Recall 很低）。

四、召回率（Recall）——"宁枉勿纵"

$$Recall = \frac{TP}{TP+FN} = \frac{27}{27+3} = 0.900$$

30 条真实负面中，模型抓到了多少？90%。

召回率 = 抓的全不全。越高 = 越不漏网。

极端例子：模型预测全部为负面 → $Recall = 30/30 = 100\%$ 。但 70 条正面也全判负面了（Precision 很低）。

五、F1 —— 精确率和召回率的调和平均

F1 让 Precision 和 Recall 同时高才有高分。

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} = 2 \times \frac{0.844 \times 0.900}{0.844 + 0.900} = 0.871$$

为什么要用调和平均而不是简单平均？

Precision=1.0, Recall=0.1 → 算术平均 = 0.55 → 看起来还行?
→ 调和平均 F1 = 0.18 → 实锤：很差！

Precision=0.8, Recall=0.8 → F1 = 0.80 ← 均衡
Precision=0.9, Recall=0.7 → F1 = 0.79 ← 也不错
Precision=1.0, Recall=0.0 → F1 = 0.00 ← 全废

调和平均的特性：任何一个低，F1 就被拉得很低。防止"偏科"。

六、Macro-F1 vs Micro-F1 —— 多分类怎么办

文档反复提到 Macro-F1。假设 3 分类：

每类单独算 F1：

正面 F1 = 0.90
负面 F1 = 0.85
中性 F1 = 0.60 ← 拖后腿

Micro-F1 = 把所有类的 TP/FP/FN 加总再算一次 F1
= 被多数类主导，和 Accuracy 类似的问题

Macro-F1 = (0.90 + 0.85 + 0.60) / 3 = 0.783
= 每类权重相同，暴露了"中性"分类差的问题 ← 推荐

七、回到你的问题：训练集 F1 高、新数据 F1 低

这正是文档中说的**灾难性遗忘 + 过拟合**：

训练集 F1 = 0.95 → 模型"背"住了训练数据
验证集 F1 = 0.62 → 换一批没见过的数据，歇菜了
差距 = 0.33 → 典型的过拟合

原因	F1 的表现
数据太少，模型死记硬背	训练集各指标高，验证集各指标低
学习率太大，预训练知识被破坏	同上（灾难性遗忘）
训练集和线上数据分布不同	训练集指标正常，线上突然崩

F1 在这里的角色 = 过拟合的报警器。 只看 Accuracy 看不出问题，F1 一算训练/验证的差距，立刻暴露。

一句话总结

F1 = Precision（抓得准不准）和 Recall（抓的全不全）的调和平均。 一个高一个低就拉胯，两个都高才高分。多分类用 Macro-F1（每类平均）防止多数类掩盖少数类问题。训练集 F1 >> 验证集 F1 = 过拟合警报。

为什么小学习率很重要？——类比：

预训练模型就像一个训练有素的翻译官（已经精通英语）
微调就像让他学习"医疗领域翻译"

- 学习率太大（ $1e-3$ ）：
- 相当于让他从零开始学医疗翻译
 - 他可能把已有的英语能力都忘了（灾难性遗忘）

- 学习率合适（ $2e-5$ ）：
- 相当于在已有英语能力上，微调医疗术语
 - 保留了已有知识，同时学会了新领域

6.2 冻结微调（不推荐）

- 固定 BERT 所有层的参数，只训练顶部分类头
- 效果全面弱于全量微调
- 仅在显存极度受限（如边缘设备）才作为妥协方案

全量 vs 冻结效果对比（典型数据）：

任务：中文情感分类（SST-2风格）

全量微调：Accuracy = 94.2%

冻结微调：Accuracy = 87.5%

差距：6.7个百分点

- 原因：冻结时BERT的通用表示无法适配任务分布
- "差"在通用语境和情感语境中的含义可能不同
 - 全量微调允许BERT内部调整，冻结则不行

七、微调工程实践要点

7.1 学习率设置

参数	推荐值	说明
BERT 层学习率	$2e-5 \sim 5e-5$	过大会破坏预训练知识
分类头学习率	$1e-3 \sim 5e-3$	新初始化的参数需要更大学习率
学习率调度	warmup + linear decay	防止训练初期震荡

💡 新手注：warmup + linear decay 是什么？

- **warmup（预热）**：训练开始时，学习率从0逐渐升高到目标值（比如前10%步数），而非一上来就用 $2e-5$ 。原因：模型刚加载预训练权重，参数处于"预训练状态"，突然大步更新会破坏已学知识。warmup让模型"慢慢起步"，平稳过渡。
- **linear decay（线性衰减）**：过了warmup阶段后，学习率从目标值线性降到0。原因：训练后期模型已经接近最优，大步更新反而会震荡，逐步缩小步长让模型收敛到更好的位置。
- 整个曲线像一座山：先爬上去，再慢慢滑下来。相当于找函数的最低点，控制步长。

差异化学习率代码示例：

```
from transformers import AdamW, get_linear_schedule_with_warmup
```

```
# 分组设置学习率
optimizer_grouped_parameters = [
    {'params': [p for n, p in model.bert.named_parameters()],
     'lr': 2e-5},          # BERT层: 小学习率
    {'params': [p for n, p in model.classifier.named_parameters()],
     'lr': 1e-3},          # 分类头: 大学习率
]

optimizer = AdamW(optimizer_grouped_parameters)
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=100,    # warmup步数
    num_training_steps=1000  # 总步数
)
```

7.2 训练超参

参数	推荐值	说明
Batch size	16 ~ 32	显存不足时用梯度累积
训练轮数	3 ~ 5 epochs	过多易过拟合
最长序列长度	128 或 512	取决于输入分布

💡 新手注：Batch Size、Epoch、梯度累积到底是什么？

- **Batch Size**：每次同时喂给模型的样本数。越大训练越稳定，但显存占用越高。16~32是BERT微调的甜点。
- **Epoch**：整个训练集过一遍叫1个epoch。3~5个epoch意味着训练数据被模型看了3~5遍。太多了模型会“背住”训练数据=过拟合。
- **梯度累积**：显存不够用大batch时的“作弊方法”——每次用小batch算梯度但不更新参数，累积几步后一起更新，效果等价于用大batch。

梯度累积示例：

```
# 目标batch_size=32, 但显存只够batch_size=8
accumulation_steps = 4  # 累积4步再更新

optimizer.zero_grad()
for i, batch in enumerate(data_loader):
    outputs = model(**batch)
    loss = outputs.loss / accumulation_steps  # 缩放loss
    loss.backward()

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()          # 累积4步后更新
        optimizer.zero_grad()     # 清空梯度
```

7.3 常见工程坑

坑	说明	正确做法
数据未 shuffle	模型先见所有负样本后见正样本	<code>DataLoader(shuffle=True)</code>
忘记 attention_mask	PAD token 参与了计算	始终传入 <code>attention_mask</code>
tokenizer 截断长度不一致	训练512，线上128	统一 <code>max_length</code> 参数

attention_mask 遗忘的后果示例：

```
# 错误：不传attention_mask
outputs = model(input_ids=input_ids)
# PAD token的向量会参与[CLS]的计算 → 语义被噪声污染

# 正确：传入attention_mask
outputs = model(input_ids=input_ids, attention_mask=attention_mask)
# PAD token被mask掉 → [CLS]只基于有效token计算
```

7.4 防止过拟合

方法	说明	代码示例
Dropout	BERT 默认 0.1	<code>nn.Dropout(0.1)</code>
数据增强	回译、同义词替换	见下文
早停	验证集 F1 连续 N 轮不提升则停止	见下文

早停代码示例：

```
best_f1 = 0
patience = 3 # 连续3轮不提升则停止
counter = 0

for epoch in range(max_epochs):
    train_loss = train(model, train_loader, optimizer)
    val_f1 = evaluate(model, val_loader)

    if val_f1 > best_f1:
        best_f1 = val_f1
        counter = 0
        save_model(model) # 保存最佳模型
    else:
        counter += 1
        if counter >= patience:
            print(f"早停于第{epoch}轮，最佳F1={best_f1:.4f}")
            break
```

八、LLM 提示词分类（无需训练）

8.1 核心思想

将分类视作生成任务，由 prompt 描述类别和规则，让大模型直接输出标签——零标注、零训练。

8.2 工作流程对比

步骤	BERT 微调（需训练）	LLM Prompt（无训练）
1	收集标注	写 Prompt
2	训练模型	调 API
3	部署模型	解析输出
4	在线推理	—

时间成本对比：

BERT 微调上线： 收集标注(2周) → 训练(3小时) → 部署(1天) = 约2.5周 LLM Prompt 上线： 写Prompt(10分钟) → 调API(5分钟) → 解析输出(30分钟) = 约1小时

8.3 Prompt 模板示例（情感三分类）

你是一个文本分类助手。 请将输入文本归为以下类别之一： [正面，负面，中性] 规则： - 只输出类别名，不要解释 - 无法判断时输出「中性」 文本：这家店服务真的太差了 类别：

Prompt 设计三要点：

要点	说明	好的示例	坏的示例
类别枚举完备	列出所有可能的类别	[正面, 负面, 中性]	[正面, 负面]（缺少中性）
明确边界规则	说明模糊情况如何处理	"无法判断时输出中性"	不说明边界情况
限制输出格式	约束输出格式	"只输出类别名"	不限制（可能输出"我认为是正面"）

九、LLM SFT 微调

9.1 核心思想

SFT (Supervised Fine-Tuning): 用「指令-回答」对继续训练 LLM，让它学会按要求输出特定格式的标签。

9.2 数据格式: Alpaca 风格

```
[
{
  "instruction": "判断以下评论的情感（正面/负面/中性）",
  "input": "快递太慢了，差评",
  "output": "负面"
},
{
  "instruction": "判断情感",
  "input": "产品超出预期",
  "output": "正面"
}
]
```

9.3 训练流程详解

第1步：模板拼接

```
instruction: "判断以下评论的情感（正面/负面/中性）"  
input: "快递太慢了，差评"  
output: "负面"
```

拼接后的prompt:

"Below is an instruction that describes a task, paired with an input.

Instruction:

判断以下评论的情感（正面/负面/中性）

Input:

快递太慢了，差评

Response:

负面"

第2-3步: Tokenize + Loss 掩码

完整token序列: [Below is an instruction ... 负面]

 ↑ prompt部分 ↑ ↑ output部分 ↑

Label设置:

prompt部分: label = -100 ← 不计算loss

output部分: label = "负面"对应的token id $\leftarrow$ 计算loss

为什么要掩码？

如果prompt也计算loss → 模型会花大量参数学习"Below is an instruction..."

- 浪费模型容量，且可能覆盖预训练知识

只在output上计算loss → 模型只学习"给定prompt后应该输出什么"

→ 更高效、更稳定

9.4 Loss掩码代码示例

```
# 构造训练数据
prompt = "判断情感\n输入：快递太慢了\n输出： "
response = "负面"

# Tokenize
input_ids = tokenizer(prompt + response)['input_ids']
prompt_len = len(tokenizer(prompt)['input_ids'])

# 设置labels
labels = input_ids.copy()
labels[:prompt_len] = [-100] * prompt_len # prompt部分mask掉

# 训练时只有response部分计算loss
# loss = CrossEntropyLoss(model_output[prompt_len:], labels[prompt_len:])
```

十、LLM SFT vs. BERT 微调：关键差异

维度	BERT 微调	LLM SFT
模型结构	Encoder + 分类头	Decoder（无新增头）
输入格式	[CLS] + tokens + [SEP]	自然语言 prompt
标签形式	整数索引（0/1/2...）	自然语言文字（正面/负面）
训练目标	分类头的 CrossEntropy Loss	输出 token 的 LM Loss
参数规模	~ 110M（BERT-base）	~ 7B+（Llama / Qwen）
标签扩展	新增类需重训分类头	新增类只需补充指令样本
输出方式	概率分布，确定性强	生成式，需解析 + 兜底

具体差异示例：

同样的情感分类任务：

BERT微调：

输入：[101, 791, 3358, ...] ← token IDs

输出：[0.1, 0.8, 0.1] ← [正面, 负面, 中性]的概率分布

预测：argmax → 1 → 查表 → "负面"

特点：确定性输出，永远返回合法类别

LLM SFT：

输入："判断情感：快递太慢了" ← 自然语言

输出："负面" ← 生成文字

特点：可能输出"这个是负面的"或"负面情感"

需要：后处理解析 + 兜底逻辑

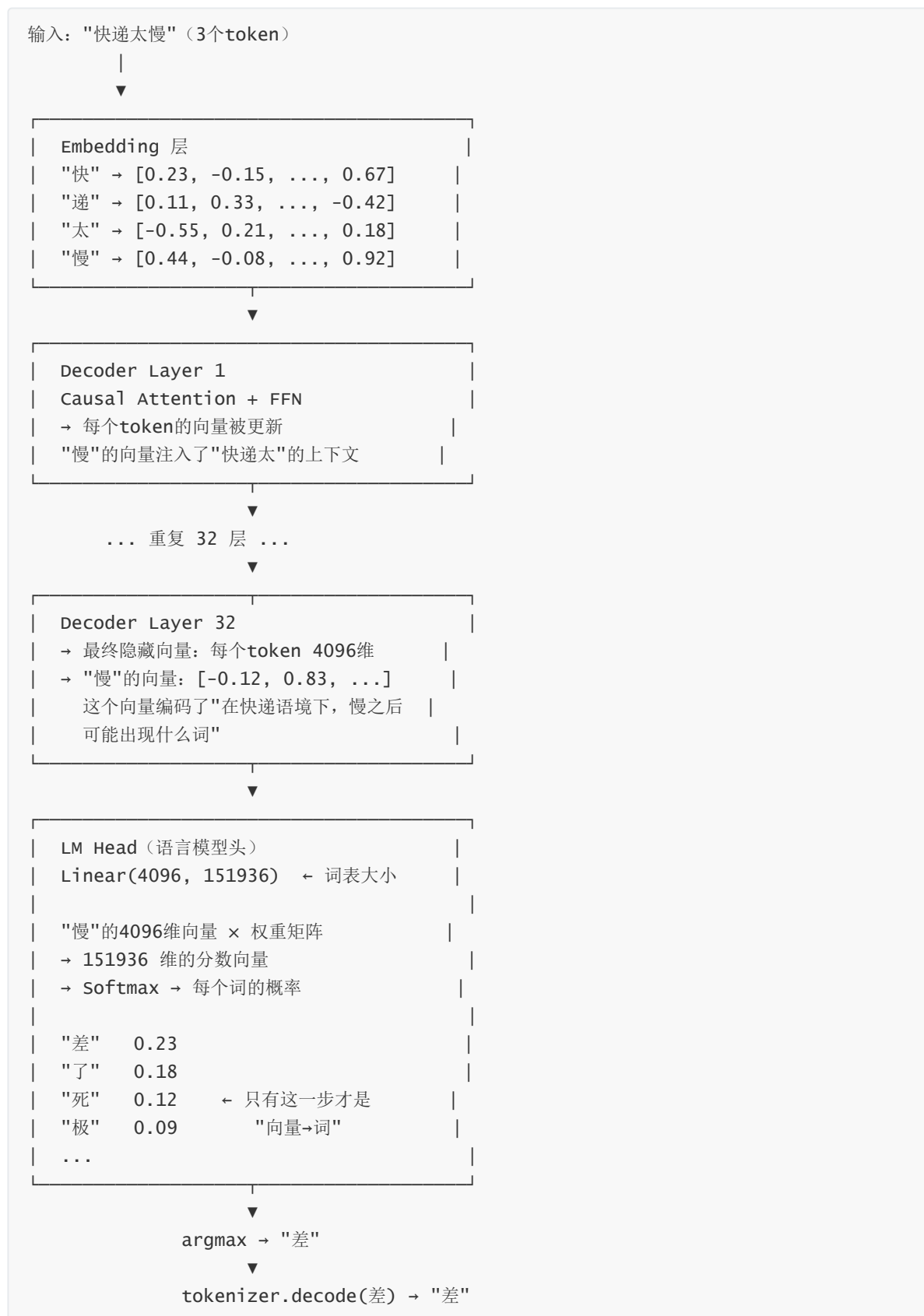
Decoder 可以理解为"向量 → 下一个词的概率"，但不只是"数值转换"

"向量数值转换成文字" —— 这个直觉对了一半。准确地说：

Decoder = 多层语义计算 + 最后一步"向量→词"的映射

真正负责"向量→文字"这一步的，是 Decoder 最顶层的 **LM Head**，不是整个 Decoder。

Decoder 内部的完整流程



三层分工

组件	做什么	输入	输出	是"数值转文字"吗
Embedding	文字 → 向量	"快"这个字	4096维向量	✗ 反向
Decoder 32层	语义理解、上下文融合	所有 token 的向量	每个 token 的富含语义的向量	✗ 语义计算
LM Head	向量 → 词表概率	最后一个 token 的 4096 维向量	词表大小维的概率分布	☑ 就是这步!
tokenizer.decode()	token ID → 文字	786 ("差"的ID)	"差"	☑ 纯解码

为什么说 Decoder ≠ 简单的"向量转文字"

如果 Decoder 只是做向量→文字的映射，那它只需要一层 `Linear(4096, vocab_size)` 就够了，不需要 32 层。

32 层的真正价值在于**中间的语义计算**：

输入 "快递太慢"	
第 1~8 层：	理解每个字的基本含义
第 9~16 层：	建立 "快递" 和 "慢" 之间的因果/转折关系
第 17~24 层：	激活情感知识——"太慢" 通常接负面评价
第 25~32 层：	精确化——最可能的下一个词是 "差"、"了"、"死"
LM Head：	把第32层的语义向量"翻译"成词表上的概率

类比：Decoder = 一个厨师做菜的全过程（洗、切、炒、调味），LM Head = 最后一步装盘。你不能说"做菜 = 装盘"。

另一层含义：为什么叫 Decoder

这个名字来自 Transformer 原始论文的 **Encoder-Decoder** 结构：

原始 Transformer（机器翻译）：	
Encoder →	把中文"编码"成一个向量表示
Decoder →	从向量"解码"出英文
LLM（GPT 系列）：	
只有 Decoder →	给定上文，自回归地"解码"出下文

Decoder 的本意：从某种表示中"生成"出序列。LLM 的 Decoder 就是从已生成的 token 序列中，逐步"解码"出下一个 token。

一句话

Decoder 的主体在做语义理解和上下文推理，只有最顶层那个 LM Head 才是"向量→词"的翻译器。就像思考过程 ≠ 说出答案——32 层在思考，"向量转文字"只是最后说出来那一步。

十一、LoRA：低秩分解实现高效微调

11.1 问题与解法

- **问题：**7B LLM 全量微调需 ~80GB 显存，单 A100 不够
- **LoRA 解法：**冻结原权重，只训低秩分解的小矩阵——参数量降到 0.1%

💡 **新手注：LoRA 的"低秩分解"到底是什么意思？**

"秩" (Rank) 是线性代数概念，衡量一个矩阵的"信息丰富度"。秩越低，矩阵能表达的信息越少。

LoRA 的核心洞察：微调时权重的更新量 ΔW 其实信息量很小（有效秩极低），不需要完整地存储一个 4096×4096 的矩阵，可以用两个小矩阵的乘积来近似： $\Delta W \approx B \times A$ ($4096 \times 8 \times 8 \times 4096$)。
 $r=8$ 就是"秩"，表示我们用8维来压缩4096维的信息。

类比：你要修改一幅4K高清图，不需要重新画一幅4K画，只需要在一张小纸上记下"在(x,y)位置加一笔红色"——这就是低秩分解的思想。

11.2 数学定义

$$h = W \cdot x + \frac{\alpha}{r} \cdot B \cdot A \cdot x$$

直观理解：

原始权重 W ： $4096 \times 4096 = 16,777,216$ 参数 $\rightarrow$ 冻结，不训练

LoRA添加：

A ： $4096 \times 8 = 32,768$ 参数 $\rightarrow$ 训练

B ： $8 \times 4096 = 32,768$ 参数 $\rightarrow$ 训练

LoRA总参数： $65,536 \rightarrow$ 仅为原权重的 0.39%!

数学上等价于： $\Delta W = B \times A$ ($8 \times 4096 \times 4096 \times 8 \rightarrow 4096 \times 4096$)
但我们不需要存储完整的 ΔW ，只需存储 A 和 B

初始化的巧妙设计：

$B = 0$ (零矩阵)， $A =$ 随机初始化

训练开始时： $\Delta W = B \times A = 0 \times A =$ 零矩阵
 $\rightarrow$ 模型输出与原始模型完全相同！（不破坏预训练知识）

训练过程中： B 和 A 逐渐学习， ΔW 从零开始慢慢增长
 $\rightarrow$ 模型平滑地从预训练状态过渡到微调状态

11.3 为什么 LoRA 有效？

原因	说明
内在低秩假设	微调时权重更新量 ΔW 的有效秩极低， $r=8$ 已能逼近全量效果
参数量大幅降低	7B 模型从 70 亿可训参数 $\rightarrow$ $\sim 4M$ (仅 0.06%)，显存从 80G 降到 16G
避免灾难性遗忘	原始预训练权重 W 完全冻结，模型"常识"不会被任务数据破坏
多任务零成本切换	base 模型只需一份；不同任务存独立 adapter (几 MB)，按需加载
推理无额外开销	上线时 $W' = W + BA$ 可合并回单一权重，与全量微调推理速度相同

💡 新手注：LoRA 的 target_modules "q_proj, v_proj" 是什么？

Transformer 中每个注意力层有4个投影矩阵：Q投影(q_proj)、K投影(k_proj)、V投影(v_proj)、输出投影(o_proj)。LoRA 默认只对 q_proj 和 v_proj 加低秩适配器，因为实验发现这两个最关键。如果想更强效果，可以加上 k_proj、o_proj 甚至 MLP 层，但参数量会增大。

MLP = 多层感知机，在 Transformer 里就是那个 FFN

Transformer 每层的内部结构

Transformer 一层 (Decoder 的一块)：



MLP 在 Transformer 里长什么样

```
# 这就是 Transformer 中的 MLP (也叫 FFN, Feed-Forward Network)

class TransformerMLP(nn.Module):
    def __init__(self, hidden_size=4096, intermediate_size=11008):
        # hidden_size: 输入输出维度 (如 Llama 的 4096)
        # intermediate_size: 中间膨胀维度 (通常是 4 倍, 如 11008)

        self.gate_proj = nn.Linear(4096, 11008) # 第一层: 扩到4倍
        self.up_proj = nn.Linear(4096, 11008) # 第一层副本 (SwiGLU用)
        self.down_proj = nn.Linear(11008, 4096) # 第二层: 缩回去
        self.act_fn = nn.SiLU() # 激活函数

    def forward(self, x):
        # SwiGLU 结构 (Llama/Qwen 等现代 LLM 的标配)
        gate = self.act_fn(self.gate_proj(x)) # 门控信号
        up = self.up_proj(x) # 向上投影
        return self.down_proj(gate * up) # 融合后缩回到 4096
```

直观理解：为什么叫 MLP

缩写	全称	含义
M	Multi	多层
L	Layer	每一层都是全连接
P	Perceptron	感知机 = 最简单的神经元

多层 + 全连接 + 非线性激活，三个要素一凑就是 MLP。

Transformer 中 MLP 的作用：知识存储

Self-Attention 负责**"谁和谁相关"** (关系推理), MLP 负责**"我知道什么"** (知识存储)。

类比：

Self-Attention = 图书馆的检索系统

→ "这个词和那个词有关系" → 找关系

MLP = 图书馆的书架

→ "中国首都是北京" → 存事实

研究（如 Geva et al.）发现，Transformer 的 MLP 层存储了大量**事实知识**。第一层 `Linear(hidden → 4xhidden)` 的 11008 个神经元中，有的专门激活“北京-中国”模式，有的激活“爱因斯坦-相对论”模式。

为什么 MLP 要"先放大再缩小"

4096 → Linear → 11008 → Linear → 4096

↑ ↑ ↑

输入 膨胀4倍 缩回去

为什么中间要膨胀？

原因	说明
增加容量	11008 维空间比 4096 维能存更多种"知识模式"
非线性变换	激活函数在膨胀后的空间能更好地捕捉复杂模式
瓶颈设计	输入输出维度一致，方便层层堆叠（残差连接）

回到你的问题：为什么 LoRA 加 MLP 层会更强

- 只加 Attention 层的 LoRA:
- 模型学会了"看哪里" (调整注意力模式)
 - 但"知道什么"没变 (知识存储没动)
- 加上 MLP 层的 LoRA:
- 模型不仅学会了"看哪里"
 - 还学会了"任务特有知识"
 - 效果更好, 但参数量大 (MLP 的权重矩阵更大)

LoRA 加在哪	参数量 (r=8)	效果
仅 q_proj, v_proj	~4M	基准效果
+ k_proj, o_proj	~8M	更强
+ MLP (gate/up/down)	~20M+	最强

一句话

MLP = 全连接层堆叠, 在 Transformer 里就是 Self-Attention 后面的那个 FFN。它负责存储事实知识, 先膨胀 4 倍再缩回去。LoRA 加到 MLP 上 = 不仅调整"注意力模式", 还调整"知识存储", 效果更强但参数更多。

多任务切换示例:

基础模型: Qwen2-7B (14GB磁盘空间)

任务A: 情感分类LoRA adapter → 4MB
任务B: 意图识别LoRA adapter → 4MB
任务C: 主题分类LoRA adapter → 4MB

切换方式:

加载基础模型 → 合并adapter_A → 情感分类
加载基础模型 → 合并adapter_B → 意图识别
不需要3个完整模型! 只需1个基础模型 + 3个轻量adapter

推理合并示例:

```
# 训练时:  $h = Wx + (\alpha/r)BAx$ 
# 推理时:  $W' = W + (\alpha/r)BA \rightarrow$  合并为一个权重矩阵
# 合并后推理速度与全量微调完全相同, 无额外延迟

with torch.no_grad():
    for name, param in model.named_parameters():
        if 'lora_A' in name:
            # 找到对应的B矩阵, 计算BA, 合并到W中
            W = get_original_weight(name)
            A = param.data
            B = get_lora_B(name)
            W.data += (alpha / r) * B @ A
```

11.4 PEFT 库完整代码示例

```
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM, AutoTokenizer

# 加载基础模型
base = AutoModelForCausalLM.from_pretrained(
    'Qwen2-7B',
    torch_dtype=torch.float16,
    device_map='auto'
)

# LoRA配置
cfg = LoraConfig(
    r=8,                    # 秩
    lora_alpha=16,          # 缩放因子
    target_modules=['q_proj', 'v_proj'], # 应用LoRA的模块
    lora_dropout=0.05,      # Dropout
    bias='none',           # 不训练bias
    task_type=TaskType.CAUSAL_LM,
)

# 包装模型
model = get_peft_model(base, cfg)
model.print_trainable_parameters()
# 输出: trainable params: 4,194,304 || all params: 7,615,436,800 || trainable%: 0.06%

# 训练（与普通训练完全相同）
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
for batch in train_loader:
    outputs = model(**batch)
    loss = outputs.loss
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

11.5 LoRA 关键超参

参数	推荐值	说明	调参建议
r (秩)	8/16	越大表达力越强但参数量增加	简单任务r=8, 复杂任务r=16~32
lora_alpha	r的2倍	缩放因子	α/r = 实际学习率缩放比
target_modules	q_proj, v_proj	应用LoRA的模块	想更强: 加k_proj, o_proj, MLP
lora_dropout	0.05 ~ 0.1	缓解过拟合	数据多时可设0

11.6 训练资源对比（以 7B 模型、单卡为例）

方法	显存	时长	硬件要求
全量 SFT	~ 80GB	数天	A100×8
LoRA SFT	~ 16GB	数小时	单张RTX 3090即可

Part 5 常见挑战与解决方案

一、挑战一：样本不均衡

1.1 现象

具体示例：

电商评论数据分布：
正面：5,000条（83%）
负面：800条（13%）
中性：200条（3%）

模型训练后：
Accuracy = 83%（看起来不错？）
但：模型几乎全预测为"正面"！
正面 Recall = 99%，负面 Recall = 5%，中性 Recall = 0%
Macro-F1 = (0.9 + 0.08 + 0) / 3 = 0.33（很差！）

→ Accuracy被多数类主导，掩盖了少数类的严重问题

1.2 解决方案

(1) 重采样

方法	说明	注意事项
过采样	对少数类随机复制或 EDA 增强	可能导致过拟合
欠采样	对多数类随机丢弃	小心信息损失
推荐比例	少数类/多数类 >= 1:5	—

过采样示例：

```
from imblearn.over_sampling import RandomOverSampler

# 原始分布：正面5000，负面800，中性200
ros = RandomOverSampler(sampling_strategy={
    '正面': 5000, '负面': 2000, '中性': 1000
})
X_resampled, y_resampled = ros.fit_resample(X, y)
# 新分布：正面5000，负面2000，中性1000
```

(2) 损失函数调整

Class Weight 示例:

```
import torch
from torch import nn

# 计算每个类别的权重（与样本数成反比）
class_counts = [5000, 800, 200] # 正面、负面、中性
total = sum(class_counts)
weights = [total / (3 * c) for c in class_counts]
# weights = [0.4, 2.5, 10.0]
# → 中性类的loss权重是正面类的25倍！

criterion = nn.CrossEntropyLoss(weight=torch.tensor(weights))
```

Focal Loss 详解与数值示例:

$$FL = -(1 - p)^\gamma \cdot \log(p)$$

💡 新手注: Focal Loss 的直觉——为什么叫"Focal"?

Focal = 聚焦。它让模型"聚焦"于难分类的样本, 忽略容易分类的样本。

普通交叉熵(CE)对所有样本一视同仁。但模型对多数类样本很快就学会了(预测概率 $p \approx 0.95$), 继续在这些样本上训练几乎没用。Focal Loss通过 $(1 - p)^\gamma$ 因子把易分样本的Loss压到极低

($p=0.95$ 时, $(1-0.95)^2=0.0025$, Loss被缩小400倍), 让梯度的"注意力"集中在少数类、难分类的样本上。

```
import torch
import torch.nn.functional as F

def focal_loss(logits, labels, gamma=2.0):
    ce_loss = F.cross_entropy(logits, labels, reduction='none')
    pt = torch.exp(-ce_loss) # p_t: 正确类别的概率
    focal_weight = (1 - pt) ** gamma
    return (focal_weight * ce_loss).mean()

# 数值示例
# 样本1 (易分样本): 模型预测正确类别概率 p=0.95
# CE loss = -log(0.95) = 0.051
# Focal loss = (1-0.95)^2 × 0.051 = 0.0025 × 0.051 = 0.000128
# → Focal Loss 是CE的1/400! 几乎不贡献梯度

# 样本2 (难分样本): 模型预测正确类别概率 p=0.3
# CE loss = -log(0.3) = 1.204
# Focal loss = (1-0.3)^2 × 1.204 = 0.49 × 1.204 = 0.590
# → Focal Loss 是CE的1/2, 仍贡献大量梯度

# 效果: 模型自动聚焦于难分类的样本 (通常是少数类)
```

(3) 数据增强

方法	说明	示例
回译	中文 → 英文 → 中文	"服务很好" → "The service is good" → "服务质量不错"
同义词替换	随机替换非关键词	"手机很好" → "手机不错"
EDA	随机插入、删除、交换词	"手机质量很好" → "手机质量非常好" / "手机很好"

💡 **新手注：EDA（Easy Data Augmentation）详解——为什么简单的随机操作有效？**

EDA 出自 Wei & Zou (2019) 论文，全文只有 **4 种极其简单的文本操作**：

操作	英文	比例	示例
同义词替换	Synonym Replacement (SR)	30% 的词被替换	"手机质量很好" → "手机品质很好"
随机插入	Random Insertion (RI)	10% 的词被插入	"手机质量很好" → "手机质量很好非常"
随机交换	Random Swap (RS)	10% 的词被交换	"手机质量很好" → "质量手机很好"
随机删除	Random Deletion (RD)	10% 的词被删除	"手机质量很好" → "手机很好"

为什么简单的随机操作有效？

- 模型学习的是"语义"而不是"特定字面表达"——"质量"和"品质"相似，模型应学会忽略这种字面差异
- 每次输入略有不同 → 模型被迫关注核心词汇，不容易"背死"某一种表达方式
- 论文实验：每类 500 条以下时，EDA 的效果 ≈ 多拿 **3 倍以上的标注数据**

"简单"是它的优点也是局限：

- ☒ 代码不到 50 行，零成本集成
- ☒ 对小数据集有显著提升（F1 提升 3~8 个点）
- ☒ 可能会引入语法错误（"手机质量很好"→"质量手机很好"不通顺但语义不变）
- ☒ 对长文本效果有限（因为只动了少量词，长文本改动比例太小）

回译代码示例：

```
from translators import translate

def back_translate(text, src='zh', mid='en'):
    """回译数据增强"""
    # 中文 → 英文
    en_text = translate(text, src, mid)
    # 英文 → 中文
    zh_text = translate(en_text, mid, src)
    return zh_text

# 示例
original = "这款手机续航太差了"
```

```
augmented = back_translate(original)
# → "这款手机的电池续航能力很差"（语义相同，表达不同）
```

(4) 评估指标

指标	说明	是否推荐
Accuracy	被多数类主导，不适用于不平衡场景	✗ 避免
Macro-F1	每类 F1 取平均，各类权重相同	☑ 推荐
每类 F1	单独看每个类别的 F1	☑ 推荐
混淆矩阵	直观暴露哪些类别被忽视	☑ 推荐

混淆矩阵示例：

	预测正面	预测负面	预测中性	
实际正面	4950	30	20	
实际负面	600	180	20	← 负面800条中600条被错分为正面！
实际中性	150	20	30	← 中性200条中150条被错分为正面！

→ 清楚看到：模型严重偏向正面，负面和中性大量被错分

二、挑战二：标注数据不足

2.1 场景

新业务上线，每个类别仅有几十条标注样本，全量微调容易过拟合。

信号：验证集过拟合，训练集 F1 >> 验证集 F1。

示例：

训练集 F1 = 0.95

验证集 F1 = 0.62

→ 差距0.33，严重过拟合！

→ 模型"背"住了训练数据，但没有学到泛化能力

2.2 解决方案

(1) 预训练模型

BERT 微调本身就是最有效的少样本学习方法。

数据量对比（情感分类任务）：

从头训练LSTM：

100条/类 → F1 = 0.55

1000条/类 → F1 = 0.78

BERT微调：

100条/类 → F1 = 0.85 ← 远超LSTM用1000条的效果

1000条/类 → F1 = 0.94

→ 预训练模型大幅降低了标注数据需求

(3) 半监督学习

少量标注数据 → 训练弱模型 → 对大量无标注数据打伪标签 → 用伪标签全量训练

完整流程示例：

初始标注数据：每类50条（共150条）

第1轮：

用150条训练弱模型A

A对10000条无标注数据打伪标签

置信度>0.9的保留：得到3000条伪标签数据

用150+3000=3150条重新训练 → 模型B

第2轮：

模型B对剩余7000条打伪标签

置信度>0.9的保留：再得2000条

用150+2000=2150条重新训练 → 模型C

最终模型C的F1比模型A提升10+个百分点

(4) 主动学习

主动学习代码示例

```
import numpy as np
```

```
def uncertainty_sampling(model, unlabeled_data, n_samples=100):
```

```
    """选择模型最不确定的n_samples个样本"""
```

```
    probs = model.predict_proba(unlabeled_data)
```

```
    # 方法1：最小置信度
```

```
    uncertainty = 1 - probs.max(axis=1)
```

```
    # 方法2：边缘采样（最犹豫的两个类别之间）
```

```
    # sorted_probs = np.sort(probs, axis=1)
```

```
    # uncertainty = sorted_probs[:, -1] - sorted_probs[:, -2]
```

```
    # 方法3：熵采样
```

```
    # uncertainty = -np.sum(probs * np.log(probs + 1e-8), axis=1)
```

```
    # 选最不确定的样本
```

```
    indices = np.argsort(uncertainty)[-n_samples:]
```

```
    return [unlabeled_data[i] for i in indices]
```

```
# 主动学习循环
labeled = initial_labeled_data # 初始标注
unlabeled = large_unlabeled_pool # 大量无标注数据

for round in range(5):
    model = train(labeled)
    to_label = uncertainty_sampling(model, unlabeled, n_samples=200)
    new_labels = human_annotate(to_label) # 人工标注
    labeled.extend(new_labels)
    unlabeled = [x for x in unlabeled if x not in to_label]
```

三、挑战三：类别定义模糊

3.1 现象

标注者间一致性低 ($\text{Kappa} < 0.6$)

示例：同一条评论“还行吧”

标注者A：中性 （“还行”=一般般）
标注者B：正面 （“还行”=还不错）
标注者C：中性 （“吧”=不确定语气）

3个人2个中性1个正面，多数票→中性
但如果是2个标注者，1:1 → 无法确定！

💡 **新手注：Cohen's Kappa 为什么不直接用“一致率”？**

因为两个人随机瞎标也会有一定概率碰巧一致。比如3分类，纯随机标注的一致率也有约33%。
Kappa扣除了“随机一致”的部分，只衡量“超出随机的一致程度”：

- $\text{Kappa}=1$ ：完美一致
- $\text{Kappa}=0.7\sim0.8$ ：良好一致
- $\text{Kappa}<0.6$ ：一致性差，标注规范需要改进

类比：两个学生做选择题，全选C也能碰巧一致30%，但Kappa会告诉你这只是随机碰巧。

3.2 解决方案

细化标注规范示例：

情感分类判断树：

1. 文本是否包含明确的情感表达？
 - └ 是 → 2
 - └ 否 → 中性
2. 情感是正面还是负面？
 - └ 包含正面词汇/表达 → 3
 - └ 包含负面词汇/表达 → 4
3. 正面程度如何？
 - └ 强烈（“非常好”、“太棒了”）→ 正面
 - └ 轻微（“还行”、“凑合”）→ 中性
4. 负面程度如何？

- └─ 强烈 ("太差了"、"垃圾") → 负面
- └─ 轻微 ("一般"、"有点小问题") → 中性

Cohen's Kappa 系数数值示例：

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

3个类别（正面/负面/中性），2个标注者

标注者A vs 标注者B的一致性矩阵：

	B=正面	B=负面	B=中性
A=正面	40	2	8
A=负面	1	35	4
A=中性	5	3	30

观测一致率 $p_o = (40+35+30)/128 = 0.820$

随机一致率 $p_e = (50 \times 43 + 40 \times 40 + 38 \times 45) / 128^2 = 0.338$

$\kappa = (0.820 - 0.338) / (1 - 0.338) = 0.728 \rightarrow$ 良好一致

如果 $\kappa < 0.6 \rightarrow$ 需要改进标注规范

四、挑战四：类别动态变化

4.1 增量微调详解

原始模型：3分类（正面/负面/中性）

业务新增类别："投诉"

错误做法：

- 只用投诉数据训练 $\rightarrow$ 模型"忘记"了原来的3个类别
- $\rightarrow$ 正面/负面/中性的F1从95%降到60%（灾难性遗忘）

正确做法：

- 混合数据训练：50%新类别数据 + 50%旧类别数据
- $\rightarrow$ 旧类别F1仅下降1~2%（可接受）

💡 新手注：增量微调为什么必须混合旧数据？

神经网络是"用进废退"的——如果训练数据中完全没有了某个类别的样本，模型就会逐渐"忘记"怎么识别那个类别。混合旧数据就是为了在学新知识的同时"复习"旧知识，防止遗忘。这和人类学习一样：学新课时也要定期复习旧课。

💡 新手注："向量检索替代分类"是什么思路？

传统分类模型必须知道所有类别才能训练，新增类别就要重新训练。向量检索的思路完全不同：把每个类别用几个代表性样本的Embedding平均值作为"原型向量"，分类时计算输入文本的Embedding和哪个原型最接近。新增类别只需加几条样本的Embedding，无需重训模型。适合类别频繁变化的场景。

$\rightarrow$ 新类别F1稳步提升

4.2 向量检索替代分类详解

```
python
import faiss
import numpy as np

# 1. 用Embedding模型将文本转为向量
def get_embedding(text):
    # 使用sentence-transformers等模型
    return model.encode(text) # 768维向量

# 2. 为每个类别建立原型向量（取该类所有样本的均值）
category_prototypes = {}
for category in categories:
    texts = get_texts_for_category(category)
    embeddings = [get_embedding(t) for t in texts]
    category_prototypes[category] = np.mean(embeddings, axis=0)

# 3. 构建faiss索引
dim = 768
index = faiss.IndexFlatIP(dim) # 内积相似度
vectors = np.array(list(category_prototypes.values())).astype('float32')
labels = np.array(list(category_prototypes.keys()))
index.add(vectors)

# 4. 新类别只需加原型向量，无需重训！
def add_new_category(category_name, sample_texts):
    embeddings = [get_embedding(t) for t in sample_texts]
    prototype = np.mean(embeddings, axis=0).astype('float32')
    index.add(prototype.reshape(1, -1))
    # 完成！无需重新训练任何模型

# 5. 分类推理
def classify(text):
    vec = get_embedding(text).astype('float32').reshape(1, -1)
    scores, indices = index.search(vec, k=1)
    return labels[indices[0][0]]
```

五、常见挑战速查表

挑战	信号（如何发现）	推荐解决路径
样本不均衡	Macro-F1 低，少数类 Recall 趋近 0	Focal Loss + 过采样 + 数据增强
标注数据不足	验证集过拟合，训练集 F1 >> 验证集 F1	预训练微调 + 半监督伪标签 + 主动学习
类别边界模糊	标注一致性差，人工判断也不确定	增加"其他"类 + 置信度阈值 + 人工复核
新类别频繁增加	重训代价高，上线周期 > 1 周	增量微调 / 向量检索替代分类器

通用建议：先用简单 baseline 暴露问题 → 对症下药 → 迭代优化；不要一开始就堆所有技巧。

Part 6 大模型时代的文本分类

一、Prompt Engineering: 零样本 & 少样本

1.1 零样本 (Zero-shot)

核心思想：直接在 prompt 中描述任务和类别，让大模型输出类别标签，无需任何训练或示例。

💡 **新手注：Zero-shot 为什么能做到"无需训练"？**

大模型在预训练时已经见过海量文本，其中包含大量"分类""判断"模式的文本。当你写一个 Prompt 说"请分类为正面/负面/中性"时，模型并不是真的在"学习"新任务，而是在"激活"它预训练时已经学会的模式匹配能力。就像你问一个博学的人"这句话是正面还是负面"，他不需要训练就能回答——因为他已经具备了这个能力。

完整示例：

Prompt：

```
| 你是一个文本分类助手。 |
| 请将以下文本分类为： |
| 正面 / 负面 / 中性 |
| |
| 文本：这个产品质量很好，很满意 |
| 类别： |
```

大模型输出：正面

输出不可控的示例：

同一个Prompt，大模型可能输出：

"正面"	← 符合预期
"这个是正面的"	← 多了"这个是..."的"
"我认为是正面"	← 多了"我认为是"
"Positive"	← 输出了英文

→ 需要后处理解析 + 兜底逻辑

1.2 少样本 (Few-shot)

核心思想：在 prompt 中提供 K 个输入-标签示例 (K=3~8)，大模型从中学习分类模式 (In-context Learning)。

完整示例：

Prompt：

```
| 文本分类任务：将文本分为正面/负面/中性 |
| |
| 示例1: |
| 文本：服务太差了 → 负面 |
| |
| 示例2: |
| 文本：物流很快，满意 → 正面 |
| |
| 示例3: |
| 文本：一般般吧 → 中性 |
```

文本：包装有点破损 →

大模型输出：负面（理解了"破损"是负面特征）

示例选取质量的影响——对比实验：

好的示例（多样化、有代表性）：

示例1: "太棒了" → 正面（强烈正面）

示例2: "一般般" → 中性（轻微正面）

示例3: "太差了" → 负面（强烈负面）

→ 测试准确率：85%

差的示例（同质化）：

示例1: "很好" → 正面

示例2: "不错" → 正面

示例3: "满意" → 正面

→ 测试准确率：60%（模型没学到负面和中性的模式）

二、大模型 vs. BERT 微调：如何选型

成本计算示例：

场景：每天10万次推理请求

方案A：BERT微调（本地部署）

硬件：1张RTX 3090（1.5万元）

推理速度：10ms/条

月电费：约500元

月总成本：约500元（硬件折旧另算）

方案B：大模型API

单次调用：0.01元（GPT-3.5级别）

日成本：10万 × 0.01 = 1000元

月成本：3万元

年成本：36万元

→ 高频场景，BERT微调的成本优势巨大（差60倍+）

三、In-context Learning 原理

3.1 什么是 In-context Learning

In-context Learning (ICL) 是大模型的一种涌现能力：无需更新任何参数，仅通过在输入中提供示例，模型就能"学会"任务模式。

维度	传统监督学习	In-context Learning
参数更新	是（梯度下降）	否（仅前向推理）
训练数据	大量标注	几个示例
学习方式	修改权重	激活已有知识
适用模型	任意	大模型（涌现能力，通常>7B参数）

💡 **新手注：“涌现能力”是什么？**

涌现（Emergence）= 量变引起质变。当模型参数规模从1B→7B→70B增长到某个临界点时，模型突然获得了小模型完全没有的能力（如ICL、推理、指令遵循）。这些能力无法通过训练小模型获得，只有模型大到一定程度才会“涌现”出来。目前学界对涌现的机理尚无定论，但实验现象已被广泛验证。

💡 **新手注：ICL 和 Few-shot 的关系？**

ICL 是机制（通过上下文中的示例学习），Few-shot 是场景（提供几个示例）。Few-shot 是 ICL 最典型的应用方式。实际上 ICL 的范围更广——Zero-shot 也可以算 ICL 的一种（Prompt本身就是上下文，只是没有示例）。

3.2 为什么 ICL 有效

大模型预训练时见过大量类似任务模式：

"将以下文本分类为..." → 模型见过数百万次类似的指令

"示例1：..." → 模型见过大量"从示例中学习"的模式

"类别：" → 模型学会了在冒号后输出类别名

本质：ICL不是"学习"新知识，而是"激活"模型已有的模式匹配能力

→ 模型参数不变，但不同的prompt激活了不同的知识区域

→ 类似于：你问一个专家不同的问题，他会调取不同的知识来回答

四、大模型分类的工程实践要点

4.1 输出格式控制

```
import re

def parse_classification_output(raw_output, valid_labels):
    """解析大模型输出，确保返回合法类别"""

    # 1. 精确匹配
    raw_output = raw_output.strip()
    if raw_output in valid_labels:
        return raw_output

    # 2. 包含匹配（处理"我认为是正面"这类输出）
    for label in valid_labels:
        if label in raw_output:
            return label

    # 3. 正则匹配
    pattern = r'类别[为: :]\s*([^\s, 。 ]+)'
```

```

match = re.search(pattern, raw_output)
if match and match.group(1) in valid_labels:
    return match.group(1)

# 4. 兜底：返回默认类别
return "中性" # 默认类别

# 示例
valid_labels = ["正面", "负面", "中性"]
print(parse_classification_output("正面", valid_labels)) # "正面"
print(parse_classification_output("我认为是正面", valid_labels)) # "正面"
print(parse_classification_output("这个很难说", valid_labels)) # "中性"（兜底）

```

4.2 置信度评估

💡 **新手注：置信度（Confidence）到底是什么？**

一句话：置信度 = 模型对自己预测结果的"把握程度"，用 0~1 之间的概率值表示。

数值示例：

输入: "这个手机续航太差了"

Softmax 输出:
 正面: 0.02, 负面: 0.95, 中性: 0.03
 ↑ 预测为"负面", 置信度0.95 → 很确定

输入: "还行吧"

Softmax 输出:
 正面: 0.38, 负面: 0.22, 中性: 0.40
 ↑ 预测为"中性", 但置信度只有0.40 → 在猜

置信度的业务用途：

置信度区间	含义	业务处理
> 0.9	模型很确定	直接采纳，自动处理
0.6 ~ 0.9	有一定把握	采纳但标记，定期抽查
< 0.6	不确定	转人工复核或走大模型兜底

为什么重要：模型不可能 100% 正确。低置信度 = 模型在"猜"，这个"猜"的结果风险最高。合理的系统不是信任模型的每一次输出，而是根据置信度决定"这次可以信多少"，并自动转人工或兜底。

```

import openai

def classify_with_confidence(text, valid_labels, n_samples=5):
    """多次采样评估置信度"""
    counts = {label: 0 for label in valid_labels}

    for _ in range(n_samples):
        response = openai.ChatCompletion.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": f"分类: {text}"}],

```

```
        temperature=0.7 # 一定随机性
    )
    label = parse_classification_output(
        response.choices[0].message.content, valid_labels
    )
    counts[label] += 1

    # 置信度 = 出现次数 / 采样次数
    confidence = max(counts.values()) / n_samples
    predicted = max(counts, key=counts.get)

    return predicted, confidence

# 示例
label, conf = classify_with_confidence("还行吧", ["正面", "负面", "中性"])
# 可能结果: ("中性", 0.6) → 置信度不高, 建议人工复核
```

4.3 成本优化：蒸馏示例

蒸馏流程：

1. 用大模型（GPT-4）对10万条无标注数据打标签
→ 成本：10万 × 0.03元 = 3000元
2. 用这10万条伪标签数据训练BERT-base
→ 成本：3小时GPU训练 ≈ 50元
3. 部署BERT模型处理线上请求
→ 成本：本地推理，几乎为0

总成本：3050元（一次性）vs 持续调用大模型API每月3万元

→ 1个月即可回本，长期节省巨大

五、大模型文本分类的最佳实践

5.1 推荐 workflow

第1步：Zero-shot验证（1小时）

写Prompt → 测试50条 → 准确率85%
→ 确认任务可行

第2步：Few-shot提升（2小时）

选5个高质量示例 → 准确率提升到90%
→ 确认上限足够高

第3步：收集标注（1~2周）

从大模型不确定的样本开始标注（主动学习思路）
→ 收集1000条高质量标注

第4步：训练BERT模型（3小时）

用标注数据微调BERT → 准确率95%
→ 满足生产要求

第5步：部署上线（1天）

部署BERT模型，延迟<20ms
→ 替代大模型API，成本降低60倍

5.2 避坑指南

坑	说明	正确做法
过度依赖大模型	大模型适合冷启动，不适合生产环境高频调用	冷启动用大模型，生产用轻量模型
忽略输出解析	大模型输出格式不稳定	必须有兜底解析逻辑
忽略延迟	大模型推理 > 500ms	实时场景必须用轻量模型
不做 A/B 测试	切换方案前必须做线上对比	灰度发布，A/B测试验证

核心概念速查

💡 新手注：速查表的使用方法

以下表格是"查表用"的——当你遇到具体问题时，快速查到应该用什么。但每个选择背后都有原理，建议结合文档正文理解"为什么"。

分类任务类型

类型	输出层	损失函数	典型场景
单标签	Softmax	CrossEntropyLoss	情感分类、意图识别
多标签	Sigmoid	BCEWithLogitsLoss	新闻多标签、评论属性

句子向量获取方式

方式	适用模型	推荐场景
[CLS] 向量	BERT 原生	短文本、单标签分类
Mean Pooling	RoBERTa/DeBERTa	长文本、语义相似度
Max Pooling	通用	需要突出关键特征

微调方式选型

方式	标注量	显存	输出可控性	推荐场景
Prompt（无训练）	0	无	低	快速验证、类别频繁变
BERT 全量微调	百条+	单卡	高	生产环境、延迟敏感
LLM SFT	千条+	多卡	高	已有 LLM 基建
LoRA SFT	千条+	单卡	高	单卡训练、多任务部署

一句话总结：文本分类从规则方法演进到大模型 Prompt，核心驱动力是**降低对人工特征工程和标注数据的依赖**。当前，BERT 微调是生产环境的主流方案，大模型 Prompt 是冷启动和快速验证的利器。在 Agent 时代，轻量分类模型作为基础设施，承担路由调度和安全过滤的关键角色。